

AI Course

Capstone Project Final Report

For students (instructor review required)

© 2025 SAMSUNG. All rights reserved.

Samsung Electronics Corporate Citizenship Office holds the copyright of this document.

This document is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.

To use this document other than the curriculum of Samsung Innovation Campus, you must receive written consent from copyright holder.

Bayan: On-Device Arabic Text Simplification for Readers with Dyslexia

تبسيط · Simplify

05/10/26

Cogni

Marwan Elamami
Abdulrahman Khengari
Ahmed Alaeb
Sanad Ali
Mohammed Thabet
Abdul Majid Mraied

Content

1. Introduction

- 1.1. Background Information
- 1.2. Motivation and Objective
- 1.3. Members and Role Assignments
- 1.4. Schedule and Milestones
- 1.5. Related Work

2. Project Execution

- 2.1. Data Acquisition
- 2.2. Training Methodology
- 2.3. Workflow
- 2.4. System Design
- 2.5. BayanBench
- 2.6. Human Rating Protocol
- 2.7. The Meaning Judge

3. Results

- 3.1. Data Preprocessing
- 3.2. Exploratory Data Analysis (EDA)
- 3.3. Modeling
- 3.4. Human Evaluation
- 3.5. BayanBench Results
- 3.6. User Interface
- 3.7. Testing and Improvements
- 3.8. On the Device

4. Projected Impact

- 4.1. Accomplishments and Benefits
 - 4.2. Future Improvements
 - 4.3. Limitations
 - 4.4. Conclusion
- References

5. Team Member Review and Comment

6. Instructor Review and Comment

Abstract. Arabic text can be hard to read for many reasons: clauses chain across whole paragraphs, clitics fuse into long words, and short vowels go unwritten. We present **Bayan**, an Android system that rewrites a selection of Arabic text, from inside any application and entirely on the phone, to lower its *linguistic load*: shorter clauses and fewer hard words. This is general readability, not a solution for dyslexia, whose other axes (decoding, visual layout, the auditory route, fluency) we do not address or have not tested. Most of the work went into evaluation. On SAMER, the main human-written Arabic simplification corpus, copying the input scores 77.5 SARI, and a fine-tuned AraT5v2 that beats copying on the sentences a human changed still loses to it overall; building our training mix exposed that 9.3% of a public Arabic simplification dataset overlaps the standard test sets. We introduce **BayanBench** (1,981 items split by document), which scores meaning with an open model (Gemma 4 31B) and simplification, restraint and correctness with code. Five raters and a reader with dyslexia rated 290 outputs and comparisons: the meaning scorer separates the outputs they judged faithful from the rest with an AUC of 0.85, and only meaning was rated consistently enough to validate a judge, so the benchmark keeps the measures that track people. On it, models trained on a 22,733-pair mix lower linguistic load at a cost to meaning: the medium strength setting of AraT5v2 cuts 6.3 words from the longest clause and keeps the meaning in 59.5% of outputs. A second corpus written by Gemma 4 31B (14,975 rows) trains a third model that, decoded with four beams, outperforms every earlier model: on the test split it keeps the meaning in 82% of outputs (earlier models at most 78.6%) and cuts 5.2 words from the longest clause (BayanSimplify-v0.2 in the app 1.9); even against BayanSimplify-v0.2 behind the app's newest checks it is no worse on meaning and cuts about three times as many words. BayanSimplify-v0.3 with four beams is the app's large model. A small meaning judge catches 95.5% of planted errors. On a Snapdragon 870 phone the first word of a rewrite appears 82–172 ms after the request with the int8 AraBART bundle (222 MB, the default) and 177–318 ms with the optional AraT5v2 bundles (471 MB).

Bayan at a Glance

The problem. Long clauses, rare words and missing short vowels make Arabic text hard for readers with dyslexia, and the tools that simplify text send it to a server.

What we built. Bayan is an Android app. A reader selects Arabic text in any app and chooses تبسيط (*Simplify*); a simpler version appears over the page, written entirely on the phone and offline. Bayan can also add tashkeel (the short vowels) and read the text aloud.

The models. Released on Hugging Face (huggingface.co/Congi-libya) as **BayanSimplify**: BayanSimplify-v0.1 (AraT5v2 on SAMER), BayanSimplify-v0.2 and its compact BayanSimplify-v0.2-Fast (AraT5v2 and AraBART on a 22,733-pair mix), and BayanSimplify-v0.3 (AraT5v2 on our own corpus, the best). The app downloads BayanSimplify-v0.2-Fast by default and offers BayanSimplify-v0.3 as its large model.

What it achieves.

Question	Answer
Does the best model keep the meaning?	BayanSimplify-v0.3 with four beams, the app's large model, keeps it in 82% of test outputs; every earlier model, at most 78.6%
Is the text simpler?	Its longest clause is 5.2 words shorter; the app's model before it managed 1.9
Do people agree?	Team raters found BayanSimplify-v0.3 easier on 86% of the outputs it changed (BayanSimplify-v0.2: 60%) and chose it over BayanSimplify-v0.2 in 8 of 8 comparisons; the reader with dyslexia chose it in every comparison that included it
Is it safe?	Every number is kept and no negation is flipped; a sentence that would lose either is shown unchanged
Does it run on a phone?	The fast model shows the first word in 82–172 ms and takes 222 MB; the large model writes a sentence in about 1.2 s and takes 471 MB
Tashkeel?	Added without changing a letter: 6.91% of vowels wrong outside case endings, from a 4.8 MB model
Can the numbers be trusted?	The meaning scorer agrees with human raters (AUC 0.85), and every score comes from BayanBench, an open benchmark anyone on the team can re-run

Objectives. Lower linguistic load without losing meaning: partly met. Add tashkeel without changing letters: met. Run on a phone with a default within 250 MB: met. Evidence from readers: started, with one reader with dyslexia (subsection 4.3).

What comes next. A reading study with more readers with dyslexia, measuring reading time and comprehension, and faster beam decoding, so the best model's quality also arrives word by word.

1. Introduction

1.1. Background Information

Developmental dyslexia affects an estimated 11% of Arab primary-school children, pooled across 18 studies ($N = 30,243$) [1]. It impairs reading through phonological processing, and Modern Standard Arabic adds obstacles of its own. Short vowels are normally left unwritten, so one written form carries several readings: كُتِبَ is *kataba* (he wrote), *kutiba* (it was written) or *kutub* (books), and the reader must resolve it from context. Clitics attach directly to the stem, producing long words that resist chunking, and Arabic-speaking children with reading disabilities show early deficits in exactly this morphological layer [2]. Prose chains clauses with و and ف across whole paragraphs, so one sentence can exceed working memory.

The evidence on what helps is uneven, and we build on the parts that have been measured. More frequent and shorter words sped reading and improved comprehension for readers with dyslexia in eye-tracking studies [3, 4]. Unpacking bound morphemes into separate words helped Arabic readers with reading disabilities and not their typical peers, the one Arabic finding with a tested group interaction [5]. Texts simplified by rules that split subordinate and coordinated clauses into short sentences were read faster and understood better by beginning readers, and most by the weakest of them [6]. Full diacritization, by contrast, is contested: it adds visual crowding [7], and its benefit for our readers has not been tested. We therefore treat *simplify first* as the core intervention and diacritization as an optional later stage.

1.2. Motivation and Objective

Readers meet difficult Arabic inside other applications — a message, a news page, a PDF — not in a reading app. Bayan therefore appears where the text already is: selecting text on an Android phone offers تبسيط (*Simplify*) in the system selection menu, and the rewrite opens in a panel over the same app. Every model runs on the phone, so the reader's text never leaves it and the tool works offline.

Arabic simplification data exists, but it was not enough for this task. SAMER [10], the main human-written corpus, is small and narrow: it is made of novels and teaches mostly word substitution. The other public sources are LLM-written or cover one domain. None of them was written for readers with dyslexia, and neither is our own corpus: we have no sentences written specifically for them. Dyslexia-targeted neural simplification exists for English [8], and the one Arabic dyslexia study with a language model used it as a tutor, not as an automatic rewriter [9]. Our literature searches of 17 and 23 September found no system that combines Arabic, automatic rewriting to lower reading difficulty, and on-device inference. We also chose not to target every difficulty that dyslexia brings; the scope paragraph below says which ones we address. Building the system raises three questions, which organise this report:

- RQ1** Does the standard reference-based evaluation measure what this task needs, and what does a model learn from the main human-written corpus?
- RQ2** Can one model, trained on mixed-domain data with a strength tag, lower the linguistic load of Arabic text (long clauses, hard words) without changing its meaning, and how much of that can we show with validated measures?
- RQ3** Does it run on a mid-range phone at interactive latency, and what do quantisation and the deployed input format cost?

We set four objectives against which subsection 4.1 reports the outcome:

- 01** A simplifier that beats copying the input on held-out sentences a human simplified, and that lowers linguistic load without losing meaning.
- 02** Diacritics restored on the simplified text without altering its letters.
- 03** The pipeline running on a mid-range Android phone at interactive speed, with a default simplifier within 250 MB.
- 04** Evidence from readers with dyslexia, collected with informed consent.

Scope of the claim. Bayan addresses two things: readability simplification, which lowers linguistic load in the form of long clauses and hard words, and read-aloud (text-to-speech), and it delivers both in any app. Dyslexia has other axes that this report does not address or has not tested. *Decoding*: short vowels are unwritten, we strip tashkeel before the model sees the text, and the app can add it back (a setting, off by default) but we have not tested that with readers. *Visual layout*: the app has typography settings, which we have not tested with readers. *The auditory route*: the app reads aloud, which we have not tested. *Fluency and working memory*: not measured. No reader with dyslexia has used the app itself yet; one has rated its rewrites, so objective O4 has started. We therefore make no claim that Bayan helps readers with dyslexia; what we claim is a measured reduction of linguistic load and the means to measure it.

Contributions.

- **An evaluation audit.** We show that copy-heavy test sets make SARI reward inaction, report every score beside a copy baseline and split by whether a human changed the sentence, and document that 9.3% of a public Arabic simplification corpus overlaps the SAMER and BAREC test sets (subsection 3.1, subsection 3.3).
- **A benchmark, and a check on its own measures.** BayanBench scores behaviours rather than agreement with one editor, with an open meaning scorer that anyone can rerun, document-level splits and cluster-bootstrap intervals. A human rating round showed that several of its first measures did not track what readers judged, so we dropped them and rebuilt it from measures that do (subsection 2.5, subsection 3.4).
- **A meaning judge.** A small judge, released in three public variants, that catches planted and real meaning errors at about 30 times the speed of its 31B teacher (subsection 2.7).
- **An honest meaning/simplification trade-off.** Models that lower linguistic load change meaning in a measurable share of outputs; we report that share, and the changes (strength tag, corpus v1, BayanSimplify-v0.3 and the app's code guards) that reduce it (subsection 3.3, Table 3.5).
- **An on-device system,** with int8 encoder-decoder bundles decoded token by token, or with beam search, inside an Android app and measured on a phone (subsection 2.4, subsection 3.8).

subsection 1.5 places the work in the literature. Section 2 describes the data, the models, the system, the benchmark and the rating protocol; Section 3 reports the results for each research question; Section 4 draws the conclusions and states the limitations.

1.3. Members and Role Assignments

Every member merged work into the team repository; the evidence column cites the pull requests and issues that hold it (#).

Table 1. Members, roles, and the work each delivered.

Member	Role	Delivered (evidence)
Marwan Elamami	Team lead; integration; on-device model	Product-form decision (#9); evaluation protocol and fixes (#16, #18); BayanSimplify-v0.1 diagnosis; data acquisition and leakage filtering; v0.2 recipe, data and task specifications (#27–#34); on-device engine and app; phone test; beam search and tashkeel in the app (#62, #66, #67); BayanBench and its v2 package (#48); rating site and analysis; Action Plan (#25); this report
Abdulrahman Khengari	Data lead	Validators for generated pairs (#10, #11); CAMEL readability check (#17, #20); synthetic corpus v0 of 13,072 pairs, split and published (#8, #38, #39); corpus v1 with the Gemma generator and Qwen judge, and its calibration (#44, #45); the meaning judge (#47, #49); BayanSimplify-v0.3: training, preference tuning and the int8 bundle (#48, #53); BayanBench v2 design and the open meaning scorer runs (#48); the beam-search proposal (#61); the benchmark report of 35 systems (#57, #58); the pitch deck (#50)
Ahmed Alaeb	Model training	SAMER conversion; BayanSimplify-v0.1 trained and documented (#5, #22); BayanSimplify-v0.2 (AraT5v2) trained (#28, #40); human ratings (#48)
Sanad Ali	Diacritization; co-training; presenter	Four-system diacritization benchmark; Libtashkeel selected (#6, #23); BayanSimplify-v0.2-Fast (AraBART) and the speed benchmark (#29, #35); human ratings (#48); presents the pitch
Mohammed Thabet	Android application	PROCESS_TEXT plugin, result sheet and 41 tests (#14, #21); model download pipeline, sheet states and typography settings (#31, #37); human ratings (#48)
Abdul Majid Mraied	Evaluation	Scoring and leakage scripts (#7, #15); locked-set converter, copy baselines, generate-and-score path (#19, #24, #26)

1.4. Schedule and Milestones

The work reported here ran from 13 September to 4 October 2026, ahead of submission on 5 October and the pitch on 6 October. It went through three build-and-measure cycles. BayanSimplify-v0.1 was trained, scored on locked test sets and diagnosed. The v0.2 models were designed from that diagnosis, trained on two architectures (BayanSimplify-v0.2 and BayanSimplify-v0.2-Fast), exported to the phone, tested on a device, and scored on a benchmark built for the task. The last week replaced the benchmark’s closed meaning judge with an open scorer and checked that scorer against human ratings; BayanSimplify-v0.3, trained on a second corpus, was then gated against the shipped model in the app’s own pipeline. With beam search in the app and complete scores, BayanSimplify-v0.3 outperforms every earlier model, and on 5 October it became the app’s large model. Figure 1 shows the timeline; Table 2 compares target and actual dates.

1.5. Related Work

Arabic simplification data. SAMER [10] is the main human-written multi-level Arabic simplification corpus, and it is small and narrow: 15 novels rewritten at two easier levels, each version keeping the original’s sentences by design. BAREC [11] labels 69,441 sentences with 19 human reading levels but is not parallel. Public LLM-written corpora provide several levels per source, and DAASI [13] human-verified rewrites of government and insurance text. No corpus we found

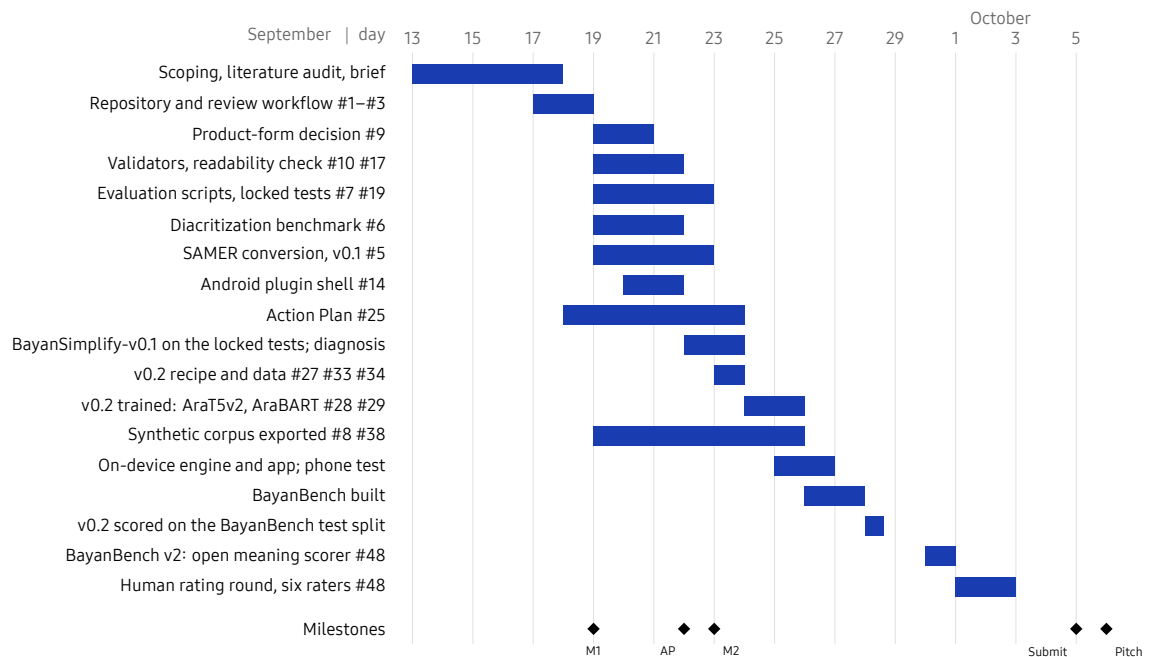


Figure 1. Timeline as executed, 13 September to 3 October. AP = Action Plan; M1, M2 = mentoring sessions. Numbers after a task are its GitHub issues.

was written for readers with dyslexia, and none teaches sentence splitting: a sentence is added in 0% of SAMER pairs, 2–4% of the LLM-written corpus’s and 8% of DAASI’s (subsection 3.2).

Controllable simplification. A control token prepended to the source lets one model rewrite for different audiences, first with target-grade tokens [15] and later with measured attributes such as length and lexical complexity [16, 17]. Our strength tags follow this line with one difference: each tag names the corpus style it was learned from, so that a meaning filter can be applied to exactly the sources that need one.

Evaluating simplification. SARI [18] is the field’s standard, but it correlates only $r = 0.36$ with human judgements of simplicity [20], and similarity-based metrics detect deleted content but not inserted or substituted content [21]. Reference-free estimates of the reading level track simplicity better [22], and edit-level error taxonomies track meaning [23]. For benchmark design we follow behavioural testing [24], construct-validity and reporting checklists [25, 26], and document-clustered intervals [27]. LLM judges are known to favour outputs close to data they wrote or approved [28], so an automatic judge must be checked against people before its scores are trusted; we report agreement between raters with Krippendorff’s α [29].

Table 2. Milestones: target against actual.

Milestone	Target	Actual
Evaluation scripts ready (#7)	Mon 21 Sep	Mon 21 Sep
Diacritizer selected (#6)	Mon 21 Sep	Mon 21 Sep
Android plugin shell (#14)	—	Mon 21 Sep
Action Plan submitted	Tue 22 Sep	Tue 22 Sep
BayanSimplify-v0.1 scored on the locked test sets	Wed 23 Sep	Wed 23 Sep
v0.2 data built; tasks issued (#28–#32)	—	Thu 24 Sep
v0.2 models trained, both architectures (#28, #29)	Fri 25 Sep	Fri 25 Sep
Synthetic corpus exported (#8, #38)	Fri 25 Sep	Sat 26 Sep
Model running on a phone, latency measured	Fri 2 Oct	Sat 26 Sep
BayanBench built; v0.2 scored on its test split	—	Mon 28 Sep
BayanBench v2: open meaning scorer; all systems scored (#48)	—	Wed 30 Sep
Human rating round complete, six raters (#48)	Fri 2 Oct	Sat 3 Oct
BayanSimplify-v0.3 ship decision (Table 3.5)	Sun 4 Oct	Mon 5 Oct: large model
Beam search in the app, a setting for BayanSimplify-v0.3 (#61, #62, #66)	—	Mon 5 Oct
Tashkeel in the app (Libtashkeel, a setting)	—	Mon 5 Oct

2. Project Execution

2.1. Data Acquisition

We acquired five corpora and produced two more (Table 3). SAMER is used under written approval from CAMEL Lab to fine-tune and to publish the resulting weights non-commercially; the corpus itself is never redistributed, and no SAMER sentence is reproduced in this report. Baseet is used with its authors’ permission. Two test splits, SAMER test and BAREC test, were locked before any training.

The synthetic corpus, v0. Our first generation pipeline took BAREC train and dev sources, drew five candidates per source from DeepSeek V4.1 Flash, and accepted one only if an LLM equivalence validator and the CAMEL BAREC readability model both passed it. The export was split by a salted hash of the normalised source, so that a sentence repeated under several BAREC identifiers never straddles two splits; BAREC test was never used. The corpus was completed after the v0.2 models had been trained, so no result in this report depends on it. Its data card and the audit that followed list what was wrong with it: tashkeel in 56.5% of rows; 63.4% of cut-off sources “completed” by the generator; 11.1% of short sources rewritten that should have been left alone; and only 42.9% of pairs at least two CAMEL levels easier. The equivalence validator, which scored 84% on 25 annotated pairs, was weaker than that number suggests: when we planted errors in its inputs it accepted 34% of added sentences and 25% of deleted ones at its threshold of 0.7. We therefore did not train on v0, and we kept it online, tagged v0, with these defects stated.

The synthetic corpus, v1. Corpus v1 replaces it on the Hub’s main branch. Gemma 4 31B writes the targets from a stricter prompt, a calibrated Qwen judge checks meaning at a threshold of 0.75, and a target must lead its source by at least two readability levels. The 14,975 rows have no violation of the rule checks, and an audit of 1,039 pairs found 1.7% with a changed meaning. It trains BayanSimplify-v0.3.

Table 3. Corpora. Sizes are rows in the files we hold. Public means on the Hugging Face Hub, where the BayanSimplify models and their on-device bundles are public too (CC BY-NC 4.0).

Corpus	Size	Licence	Used for
SAMER [10]	14,343 / 2,983 / 3,277 pairs (train / dev / test)	research; written approval	BayanSimplify-v0.1 and the v0.2 models; locked test set
BAREC [11]	54,845 / 7,310 / 7,286 sentences	CC BY-SA 4.0	locked test set; BayanBench core items; generation sources
Baseet [12]	34,294 sources, three LLM-written levels each	with permission	the v0.2 models (6,888 meaning-filtered pairs)
DAASI [13]	1,509 pairs + 350 held out	CC0	the v0.2 models (1,502 pairs); out-of-domain test
Tashkeela [14]	743 test sentences	open	diacritization benchmark
Bayan synthetic v1	14,975 rows; Gemma 4 31B generator, Qwen judge	CC BY-SA 4.0, public on the Hub	current corpus; trains BayanSimplify-v0.3
Bayan synthetic v0	13,072 pairs (12,419 generated, 563 identity, 90 scripture); DeepSeek generator	CC BY-SA 4.0, public (Hub tag v0)	superseded; not used to train any model

2.2. Training Methodology

Task. Given a source sentence x and a strength tag t , the model produces a rewrite $\hat{y} = f(t \oplus x)$ that should keep the meaning of x and be easier to read. Training minimises token-level cross-entropy on pairs $(t \oplus x, y)$; decoding is greedy, the setting the app runs for BayanSimplify-v0.2 and BayanSimplify-v0.2-Fast (BayanSimplify-v0.3 decodes with beam search in the app, subsection 2.4).

Base models. Simplification is a sequence-to-sequence task, so every candidate is a pre-trained encoder–decoder transformer (Table 4). AraT5v2 came first because it beat AraBART at all three levels in the published baselines of the corpus we add to the v0.2 training set [12]; AraBART is the on-device candidate because it is 2.6 times smaller.

Input normalisation. No vocabulary in Table 4 was built for diacritized text: in AraT5v2 only 1.6% of pieces carry a diacritic, and AraBART cannot represent one at all. Every model therefore receives its source with tashkeel and tatweel removed, in training and in the app; subsection 3.3 measures what happens otherwise.

BayanSimplify-v0.1. We fine-tuned AraT5v2 to rewrite SAMER’s level-5 sentences into their level-3 simplifications with the prefix **بسط** (*simplify*): AdamW at learning rate 10^{-4} with weight decay 0.01 and 100 warm-up steps, an effective batch of 16, mixed precision, 10 epochs (3,250 steps) and seed 42. Pairs whose simplification equals the source (42% of SAMER train) were capped at 20% of training, and the checkpoint with the best SARI on a 500-row dev subset was kept.

BayanSimplify-v0.2 and BayanSimplify-v0.2-Fast. The diagnosis of BayanSimplify-v0.1 (subsection 3.3) identified three problems, and the v0.2 recipe changes one thing for each:

- Inaction was rewarded for the wrong reason.* Capping unchanged pairs at 20% while choosing the checkpoint on a dev sample that is 43% unchanged pulls the model both ways. The v0.2 recipe keeps all 6,033 unchanged SAMER pairs, so it learns when not to edit, and selects its checkpoint on SARI over dev rows a human changed.

Table 4. Candidate base models. “Tashkeel” is measured on 1,000 diacritized BAREC dev sentences: vocabulary entries containing a diacritic, and how many more tokens the same sentences cost with diacritics than without.

	AraT5v2-base-1024	AraBART	HPLT T5 base (Arabic)
Family	T5 [35]: span-corruption pretraining, relative position buckets	BART [36]: denoising pretraining, learned positions	T5 in the NorT5 variant [37]
Layers (enc/dec), width	12/12, $d=768$, 12 heads, gated-GELU FFN 2048	6/6, $d=768$, 12 heads, GELU FFN 3072	12/12, $d=768$, 12 heads, FFN 2048
Parameters	368M (46% in two untied $110k \times 768$ embeddings)	139M	$\approx 294M$
Vocabulary	110,208 SentencePiece	50,002 SentencePiece BPE	32,768
Context	1,024 tokens	1,024 positions	512 positions
Pretraining	Arabic MSA and dialects [32, 33]	large Arabic corpora [34]	HPLT v3 web crawl, Arabic subset [38]
Tashkeel	1.6% of pieces; $2.57 \times$ tokens	0 pieces; every mark becomes <unk>; $4.53 \times$ tokens	0 pieces; $1.81 \times$ tokens
Licence	none stated on the model card	Apache-2.0	Apache-2.0
Used in	BayanSimplify-v0.1 and the v0.2 models	BayanSimplify-v0.2-Fast	not trained

- (b) *SAMER is novels, and it teaches word substitution.* The v0.2 recipe adds DAASI’s everyday and administrative text and an external corpus of educational text. The external targets are LLM-written, and many are summaries that drop facts, delete answer options or invent a speaker. An external pair is kept only if it passes a *meaning filter*: LaBSE similarity ≥ 0.646 , a target at least 67% of the source’s length, and every number kept. Each threshold is the 5th percentile of the same measure over DAASI’s human-verified rewrites, so a pair is rejected only when it departs from its source more than 95% of human simplifications do.
- (c) *One strength does not fit every reader.* Every source carries a tag naming the style it was learned from (Table 5). At run time the tag is the reader’s strength setting, so one model serves light, medium and strong rewrites [15, 16].

Table 5. The v0.2 training set (22,733 pairs). For each Baseet source, the mildest level whose target is shorter per sentence or has fewer rare or long words is used. No source overlaps any test set.

Tag	Taught by	Pairs	Meaning at run time
[S0]	SAMER L5→L3, all pairs (8,310 changed, 6,033 unchanged)	14,343	minimal: literary word swaps
[S1]	Baseet level 3, meaning-filtered	3,508	light
[S2]	Baseet level 2, meaning-filtered	1,954	medium (evaluated here)
[S3]	Baseet level 1, meaning-filtered	1,426	strong
[SA]	DAASI train	1,502	everyday and administrative style

v0.2 training. Both architectures were trained on the same data with one shared script, so that the architecture is the only difference: 10 epochs, batch 8 with 2 gradient-accumulation steps, fp16, maximum length 256, seed 42, evaluation twice per epoch with patience 3, and the checkpoint chosen on SARI over the 600 changed rows of a fixed 900-row SAMER dev sample. AraT5v2 used learning rate 10^{-4} and the prefix :بسط; AraBART used $5 \cdot 10^{-5}$ and :بسط without

the shadda, which its tokenizer maps to an unknown token. The script disables length-grouped batching, which in the pinned transformers 4.46 also shuffles the evaluation set and would score each prediction against another row's reference; it stops if predictions and references ever fall out of order, and the logs show full alignment at all 40 evaluations.

BayanSimplify-v0.3. The measurements of the v0.2 models (subsection 3.3) named restraint, and protection of text that must not change, as the open problems. BayanSimplify-v0.3 retrains AraT5v2 on corpus v1 instead of the v0.2 mix. In the app it runs behind the text step's code guards (subsection 2.4), which keep the source sentence when a rewrite loses a number or flips a negation and pass protected text through untouched. It ships as the app's large model, decoding with four beams (Table 3.5).

Figure 2 summarises how the training pipeline changed. Each step was driven by a measured problem in the one before it.

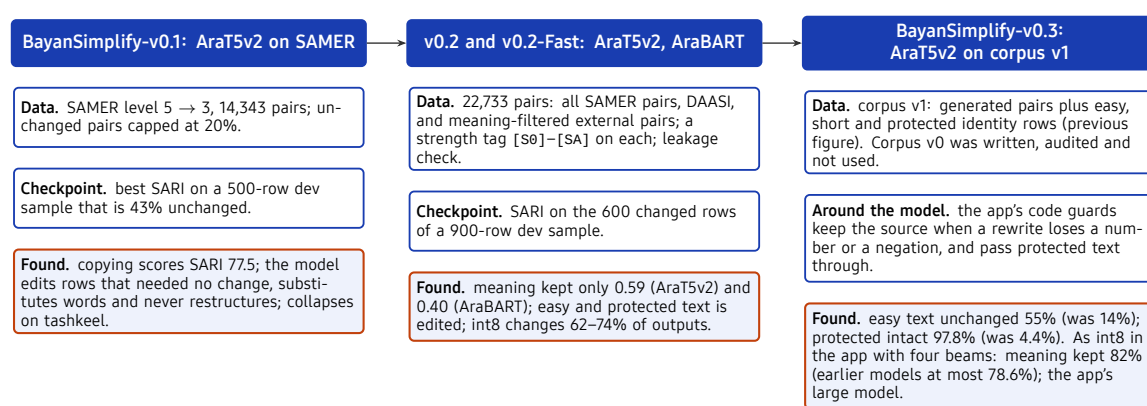


Figure 2. How the training pipeline evolved. Each model's *Found* box is the measurement that produced the next model's changes.

2.3. Workflow

Each task is a GitHub issue with an owner, a reviewer and a definition of done; changes arrive as reviewed pull requests. Over the period reported, the team opened 23 issues and 28 pull requests, of which 22 were merged, with merged work from every member. SAMER text and model outputs on SAMER move only through private Kaggle datasets and a private Hugging Face organisation, never through the repository.

Figure 3 shows the data path and its three gates. The leakage check compares every training file with both locked test sets, by exact match and by character 5-gram Jaccard overlap of 0.8 or more. The meaning filter admits LLM-written pairs only if they keep the source's meaning. Generated pairs pass validators before release. The locked test sets are fixed by row count and SHA-256 in a committed manifest and re-verified before every test run.

2.4. System Design

Figure 4 shows the system as tested on a phone. The Android app registers an activity for `android.intent.action.PROCESS_TEXT` (text/plain, Android 6.0 and later), so تبسيط appears in the selection menu of any app. A translucent activity draws the result over the host app without the overlay permission; an optional floating panel, which does require it, keeps reading while the reader returns to the page.

Inference. The simplifier ships as an int8 ONNX bundle (encoder, decoder with a key–value

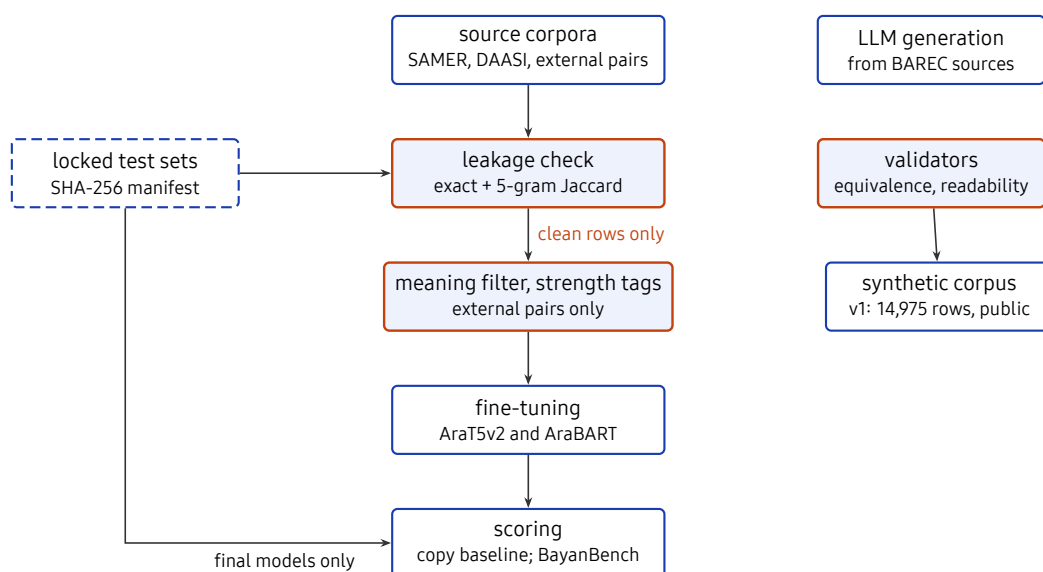


Figure 3. Data and evaluation workflow. Orange boxes are the gates that can reject data. The generated corpus (right) is a public artefact that feeds BayanSimplify-v0.3.

cache, and tokenizer), downloaded once, verified by SHA-256, and run by ONNX Runtime with a decoding loop written in Kotlin that blocks repeated 3-grams and stops at 256 tokens. BayanSimplify-v0.2-Fast (the fast model) and BayanSimplify-v0.2 decode greedily and stream each token to the screen as it is produced. BayanSimplify-v0.3, the large model, decodes with four beams (the setting *More faithful, slower*, on by default; #61, #62, #66) and shows each sentence when it is finished: a port of transformers' beam search that runs the beams as one batch and reorders the decoder's cache when a beam continues another. On the same CPU the app's engine gives transformers' exact tokens on 120 of 120 BayanBench sentences, and the search matches transformers' own on all 578 test cases whose decisions are clear of float rounding. Beams can still change earlier words until they finish, so with the setting on each sentence is shown when it is final. The app strips tashkeel, splits the selection into sentences and decodes them one by one. Sentences under six words (headings, labels) pass through unchanged, because the model was trained on longer sentences and invents text around fragments.

The app's text step. Before the model, the selection is cleaned (PDF line breaks, presentation and Persian letter forms, digits glued to letters, list bullets), Arabic-Indic and Persian digits are mapped to Western digits, and scripture and set poetry are detected (Qur'anic marks and verse brackets, the usual quotation formulas, fully vowelised verse) and *never sent to the model*: they come back exactly as selected. After the model, a sentence keeps its source wording when the rewrite keeps less than 35% of its content words, drops or changes a number, drops a Latin-script word, or changes the count of negations or of limits (*at least, only, ...*); since #66 it also checks condition words and counts Latin-script words one by one, the checks BayanSimplify-v0.3's benchmarked safety net used (#61). Digits return in the source's own style, and a selection cut mid-sentence keeps its open ending. On the BayanBench dev core items these guards raise numbers kept from 72% to 100% and protected text and cut-off endings to 100%, remove every flagged negation and limit change, and raise meaning kept by about 3.5 points without changing how much is simplified; they fall back on 3–7% of sentences. The original is always one tap away.

Read-aloud and diacritization. Voices run on the device through sherpa-onnx. The default,

Nabra-7M-Distill, is a Kokoro-architecture Arabic voice exported to int8 (8.7 MB); three Piper voices are optional downloads. The spoken word is highlighted as it is read. The diacritizer, Libtashkeel (Table 22), runs on the device too: a 4.8 MB character model bundled with the app. With *Show tashkeel* on (off by default, since full vowels can crowd a line), each finished sentence gets its short vowels after the guards, so the reader, read-aloud, copy and history all use the same text. The Kotlin port gives the Python package's exact output on 305 BayanBench texts and never changes a letter.

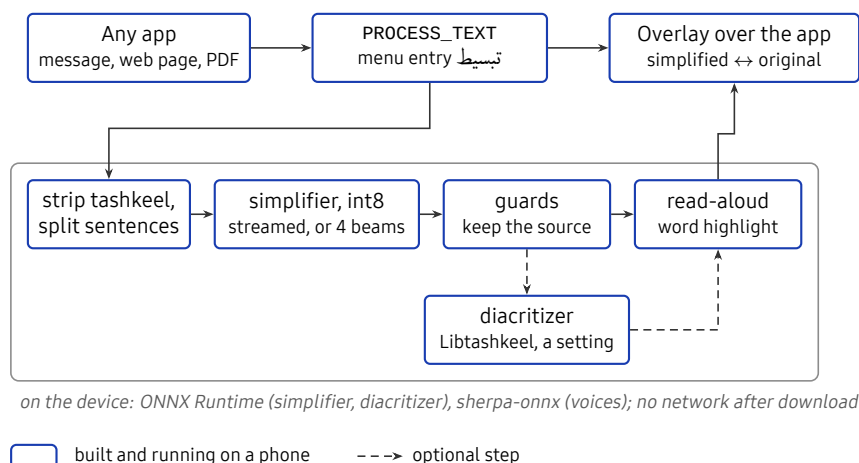


Figure 4. System architecture of the build tested on the phone. The app is entered from the host app's selection menu and answers with an overlay on top of it.

2.5. BayanBench

SARI measures agreement with one editor's rewrite. A reader with dyslexia needs something else: that a selection comes back *easier to read* and *with the same meaning*, and that what should not change does not. No public benchmark tests this for Arabic, so we built one against the construct-validity checklist of [25] and the reporting practices of [26].

Construct. A rewrite is dyslexia-friendly when it says the same thing in a form that is easier to read and follow: shorter clauses, more familiar words, no new errors, and restraint where the text is already easy. Visual formatting, read-aloud and tashkeel are out of scope, and summarising is always a failure: dropping content never counts as simplification.

Measures. Every output is scored on separate measures, grouped in blocks and never combined into one number (Table 6); the probe tracks follow CheckList [24].

- *Meaning*, from an open model (below): the mean P(same meaning), and *meaning kept*, an output whose P(same meaning) is at least 0.5 and that keeps every number of its source. An unchanged copy keeps its meaning by definition.
- *Simpler*, as changes from source to output, each on the items where there is something to simplify: words cut from the longest clause (sources with a clause over 15 words, BAREC's guideline limit for level 10, about grade 4), reading levels lowered (the maximum sentence level predicted by the CAMEL BAREC readability model, sources at level 10 or above), and hard words removed (words at levels 4–5 of the SAMER readability lexicon).
- *Checks*, deterministic: every number kept, and no house punctuation rule broken.
- *Behaviour*: easy text left unchanged (C), protected text keeping its letters (F), and a cut-off selection keeping its ending (E1).

- *Flags*: a changed count of negations, limits (at least, at most, only) or conditions. A flag marks an output for attention and is never a failure on its own, because a rewrite can rephrase a negation correctly.

A *clause* is a maximal span between punctuation marks (full stop, question and exclamation marks, comma, semicolon, colon, line break), so the clause measure credits added commas as well as split sentences and is read next to meaning. Numbers are compared as multisets after mapping Arabic-Indic and Persian digits to Western ones; negations are counted over eleven negating particles. The pass/fail measures of the first version (no clause over 15 words, at least 60% of the source’s words kept) are still computed as diagnostics.

Table 6. BayanBench tracks, with items per track in the dev / test splits.

Track	Behaviour	Measured by	Items
A1–A3	meaning kept: general; hard spots (negation, numbers, conditions, cause, quantity, certainty); high stakes (legal, medical)	meaning scorer; numbers kept; flags	299 / 508
B	long clauses shortened	words cut from the longest clause; meaning scorer	104 / 187
B2, B3	fewer hard words; lower reading level	SAMER word levels; CAMEL BAREC readability model	across items
C	easy text left unharmed	unchanged; meaning scorer	129 / 202
D	correct Arabic	house punctuation rules; human ratings	across items
E1–E4	real selections: cut-offs, messy text, 40+ words, several sentences	end mark; clause length; meaning scorer	139 / 263
F	protected text (Qur’an, Bible, classical poetry)	letters unchanged (never sent to the scorer)	60 / 90
G	on the phone	int8 bundle against the same model in PyTorch; size, time	—
H	people	290 blind rating tasks, six raters (subsection 2.6)	—

Items and splits. The 1,981 items form four sets that are reported side by side and never averaged. *Core* items come from BAREC test documents. *Held-out* items are news and legal text, including 247 BAREC sentences of those domains that no model trained on. *Outside* items are 344 sentences from UN News, WHO fact sheets, newly created Arabic Wikipedia articles, UNHCR help pages and Android help, fetched on 28 September and therefore published after every corpus we use. *Written* items are 50 medicine-leaflet sentences written for the benchmark by an LLM, kept out of core because they are not real text. Items are split by document (731 dev items from 129 documents, 1,250 test items from 197), so no document contributes to both splits. Any BAREC document with a sentence found in any training or generated data was dropped whole (17 documents), and near-copies were removed by word-trigram overlap.

The meaning scorer. Meaning is scored by Gemma 4 31B [30], run in its 4-bit quantisation-aware checkpoint with vLLM [31], as a scorer rather than a writer: it sees the source and the output, never the system’s name, and answers four yes/no questions (does the rewrite keep exactly the same meaning; does it add a fact; does it drop a fact or qualifier; does it contradict the source). The probability of yes is read from one forward pass, so no text is generated and the score is continuous. Scores are shared data keyed by the content of the (source, output) pair, so scoring a system needs no GPU, an output nobody has scored is reported as pending

rather than guessed, and a meaning rate is shown only once 90% of its items are scored. The first version of the benchmark used pass/fail verdicts from a closed model, Gemini 3.1 Pro, with a fixed checklist; the open scorer replaces it because anyone can rerun it, it costs about \$0.12 for 2,700 pairs, and it gives a graded score whose threshold can be checked against people. The 0.5 threshold was set before the human ratings and kept after them (subsection 3.4). The v0.2 training data contains nothing a Gemma model wrote or approved, and the written items come from a different model family, which limits preference leakage [28].

Statistics and release. Every rate and mean carries a 95% interval from a bootstrap over whole documents (2,000 resamples), because sentences from one document pass or fail together [27]; a comparison counts only when the interval of the paired difference excludes zero. The benchmark ships as a Python package whose command-line tool covers the whole cycle (verify, predict, score, meaning-pending, meaning), with its data, model outputs and meaning scores in a checksum-pinned Hugging Face dataset, so any team member can score a model on a laptop, on Kaggle or on Linux.

2.6. Human Rating Protocol

An automatic scorer is only useful if it agrees with people, so we rated a sample of model outputs before trusting it. The rating set holds 245 single outputs and 45 side-by-side comparisons from the dev split (core and outside items, 64 documents), drawn from the six v0.2 systems of Table 15. Copies of the input were not rated, because there is nothing to judge, and protected and constructed items (F, E1, E2) were left out. 210 outputs were drawn evenly, 35 per system; 35 more were drawn where the scorer was least sure ($P(\text{same})$ between 0.1 and 0.9, the narrowest band that held enough candidates), so the check does not rest only on easy cases. 80 outputs and 15 comparisons went to every rater, to measure agreement; the rest were split one rating each.

For each output a rater saw the source and the rewrite, without the system's name, and answered three questions: *meaning* (same, a minor change, a major change), *Arabic* (correct, a minor error, a major error) and *ease* for a reader with dyslexia (easier, the same, harder). For a comparison the question was which version is better for such a reader: easier, with the same meaning. Five team members rated on a web page built for the round, which saved every answer as it was given and bound each rater to one name. A sixth rater, a reader with dyslexia, gave informed consent and answered only the ease question, worded for them as هل كانت أسهل عليك؟ (was it easier for you?), on 30 shared outputs, and for a comparison أي النسختين كانت أسهل عليك في القراءة؟ (which version was easier for you to read?), on 10. An output's meaning counts as kept when a strict majority of its raters said *same*; a tie counts as not kept.

2.7. The Meaning Judge

Meaning is the property a simplifier most easily breaks and the one no reference-based score sees, so we trained a judge for it. It is a four-head Gemma 4 E2B model distilled from the 31B teacher that scores a (source, output) pair, released on the Hugging Face Hub in three variants (bf16, w8a8 and w4a16; CC BY-NC). It replaces the Gemini judge of BayanBench v1, whose verdicts had been validated only against themselves.

What it catches. At a 5% false-reject rate the bf16 judge flags 95.5% of planted errors and 86.5% of changes that humans labelled as meaning changes. On subtle real errors, those a rater finds only on a careful second reading, it flags about 62%. It reaches about 94% of the

teacher’s score at about 30 times the speed. The judge is not a substitute for a reader: it screens corpora and checkpoints, and the 31B model remains the reference scorer in BayanBench v2.

The human rating round. To check any judge against people, we collected ratings of 245 single outputs on three questions: whether the meaning is kept, whether the Arabic is good, and whether the text is easier. Agreement was Krippendorff’s $\alpha = 0.51$ for meaning, 0.17 for Arabic quality and 0.19 for ease. Only meaning is firm enough to validate a judge against, so the judge was validated on meaning alone. One reader with dyslexia rated ease only. The round is not a reader study: it is a small set of ratings that told us which measures to trust, and no reader has yet used the application (objective O4).

3. Results

3.1. Data Preprocessing

Conversion. SAMER’s aligned levels were converted to source–target pairs (level 5 → level 3). One converter turns each locked test file into the scoring format, keeps every row (including the 1,599 SAMER test rows, 48.8%, where the two levels are identical) and checks the row count before writing.

Normalisation for comparison. For leakage checks and scoring alignment, diacritics and punctuation are removed and the alef, ؤ/ة and ى/ي variants are unified. Training text keeps its spelling apart from tashkeel and tatweel.

Figure 5 shows the pipeline that produced the current corpus. Text is cleaned first, then each source is routed: scripture, poetry and other protected text, and headings of five words or fewer, are copied unchanged; only the remaining sources are rewritten, and a rewrite must pass four gates before it enters the corpus.

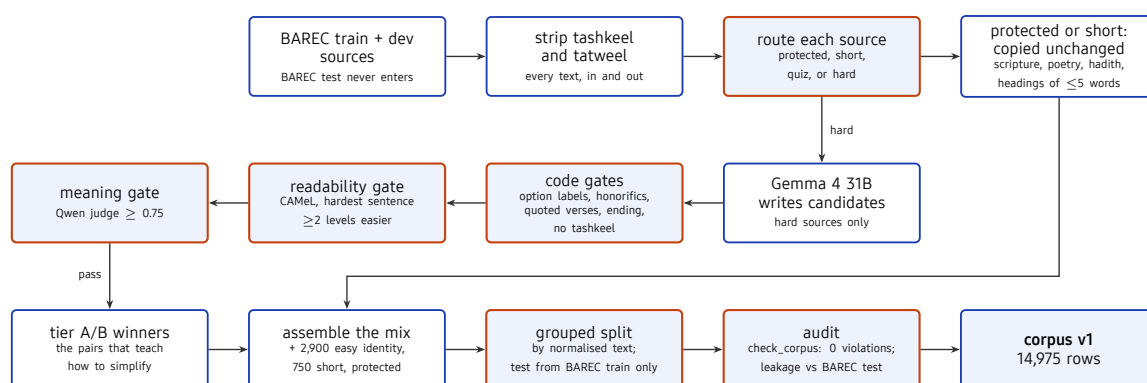


Figure 5. Data cleaning pipeline for corpus v1. Orange boxes can reject data. Counts are from `assemble_corpus_v1.py` and `check_corpus.py`.

Leakage. Every training file was checked against both locked test sets; matching rows were removed and the files re-checked until none matched. Matches under five words are reported but not counted, because short formulaic fragments recur in any corpus; BAREC alone repeats 311 sentences between its own train and test splits.

Meaning filter. Applied to the external candidates, the filter keeps 6,888 pairs across the three levels. Its purpose shows at the strongest level: the 5th percentiles of its level-1 targets are 0.44 LaBSE similarity and 26% of the source’s length, and 10% of those targets drop a number, against 0.646, 67% and none for DAASI’s human rewrites.

3.2. Exploratory Data Analysis (EDA)

Table 7. Corpus statistics, computed on the files we hold. Identity = target equals source after removing diacritics; length ratio = median of target words over source words.

Corpus	Rows	Median words	≤ 3 words	Diacritized	Identity	Length ratio
SAMER train	14,343	7	16.4%	0.0%	42.1%	1.00
SAMER test (locked)	3,277	6	23.6%	0.0%	48.8%	1.00
BAREC test (locked)	7,286	10	16.9%	53.8%	—	—
DAASI	1,509	12	8.4%	32.3%	0.0%	1.00

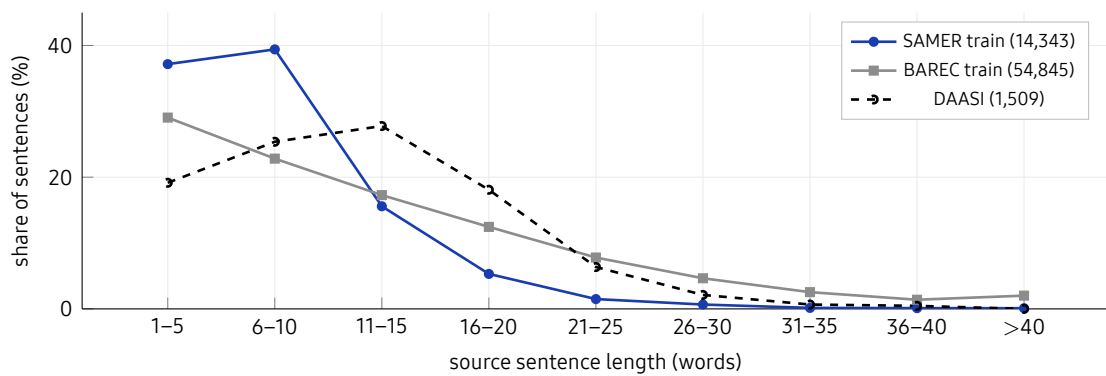


Figure 6. Source sentence length by corpus. SAMER is short literary text; BAREC reaches the long sentences readers struggle with.

Four properties of the corpora (Table 7, Figure 6) explain the behaviour of both models:

- **SAMER carries no diacritics**, while 54% of BAREC test do. A model trained only on SAMER has never seen a diacritized input.
- **SAMER simplifies words, not sentences.** Its median length ratio is exactly 1.00: the editors substituted vocabulary and kept structure.
- **No corpus teaches splitting.** A sentence is added in 0% of SAMER pairs, 2–4% of the LLM-written corpus's and 8% of DAASI's, and word length and clitic load barely change in any of them. Existing data can teach shorter, more common words; it cannot teach the restructuring the reading research supports most directly [6].
- **Half of SAMER needs no change.** 42–49% of pairs are identical, and SAMER splits by chapter rather than by novel, so its test set is in-domain. Both make SARI on SAMER read optimistically.

3.3. Modeling

Setup. Reference-based scores use SARI [18] through the `evaluate` implementation, beside BLEU, BERTScore between source and output [19], and the fall in predicted BAREC level. Every score is paired with the copy baseline and split into rows a human changed and rows a human left unchanged. Each final model was run once per test set after training and selection were complete.

RQ1: the reference-based view. Because half of SAMER needs no change, copying scores SARI 77.50 on the full test set but only 56.06 on the 1,678 rows a human changed, and the second

number is the bar a model must clear. Table 8 and Figure 7 report BayanSimplify-v0.1, with the harness validated first: it reproduced the training run’s 2,983 dev predictions row for row.

Table 8. BayanSimplify-v0.1 on test data with human references, scored once with `score.py`. SARI is split by whether the human reference changed the source (DAASI has no unchanged pairs). BERTScore compares output with source; readability drop is the fall in predicted BAREC level (positive = easier). DAASI sources and references have tashkeel stripped.

System	SARI all	SARI changed	SARI unchanged	BLEU	BERT-Score	Read-ability drop	Copy rate
SAMER test (3,277 rows, locked, in domain)							
Copy the input	77.50	56.06	100.00	72.03	1.00	0.00	100.0%
v0.1, greedy	75.88	61.83	90.62	71.68	0.98	0.08	65.3%
v0.1, beam 4	76.29	61.95	91.34	72.22	0.98	0.07	69.4%
DAASI held-out (350 pairs, government and insurance text, out of domain)							
Copy the input	45.05	45.05	—	25.25	1.00	0.00	100.0%
v0.1, greedy	43.87	43.87	—	23.66	0.99	−0.03	77.7%

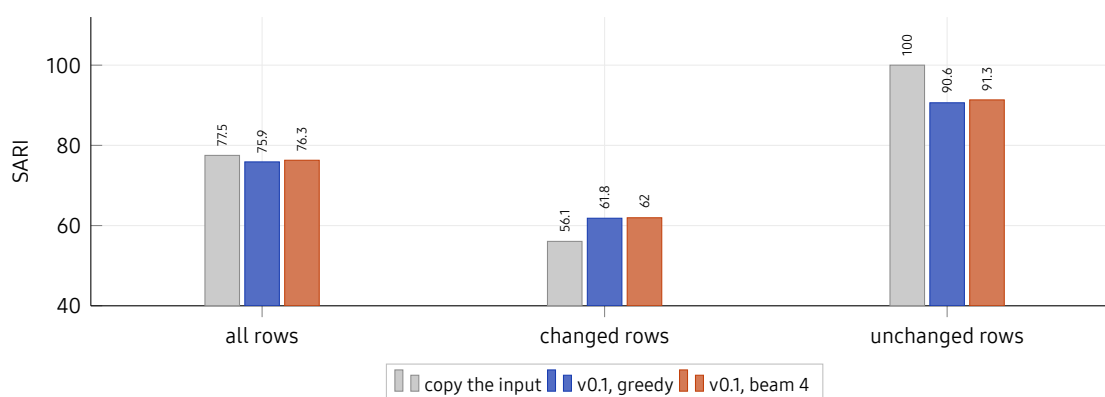


Figure 7. SARI on the locked SAMER test set by reference bucket. BayanSimplify-v0.1 clears the copy baseline where a human simplified and loses to it where a human left the sentence alone.

BayanSimplify-v0.1 simplifies where it should and over-edits where it should not. On the 1,678 test rows a human simplified, greedy decoding scores 61.83 against the copy baseline’s 56.06 (+5.77 SARI). On the 1,599 rows a human left alone it scores 90.62 against a perfect 100, because it edits 13.6% of sentences that needed no change. The two effects net to −1.62 overall, so the headline number would report a working model as worse than doing nothing. Beam search changes little (61.95 on changed rows).

What BayanSimplify-v0.1 learned. The model returns 65.3% of test rows unchanged where the editors left 48.8%, and leaves 45.3% of the rows an editor rewrote untouched. When it edits, 75.4% of its edits change one word (editors: 49.5%), and its median output-to-input length ratio is 1.00: it substitutes vocabulary and never restructures (Figure 9a). Figure 8 shows the role of checkpoint selection: the SARI that chose the checkpoint had been flat within noise since step 1,800, while the copy rate was still falling.

Diacritized and out-of-domain input. On BAREC’s 3,918 test sentences that carry tashkeel, raw input keeps 63.7% of the source words and 31.9% of outputs lose more than half the sentence; with tashkeel stripped the same sentences keep 94.6% and 1.8% lose more than half (Table 9, Figure 9b). On DAASI’s held-out government and insurance text, BayanSimplify-v0.1 scores

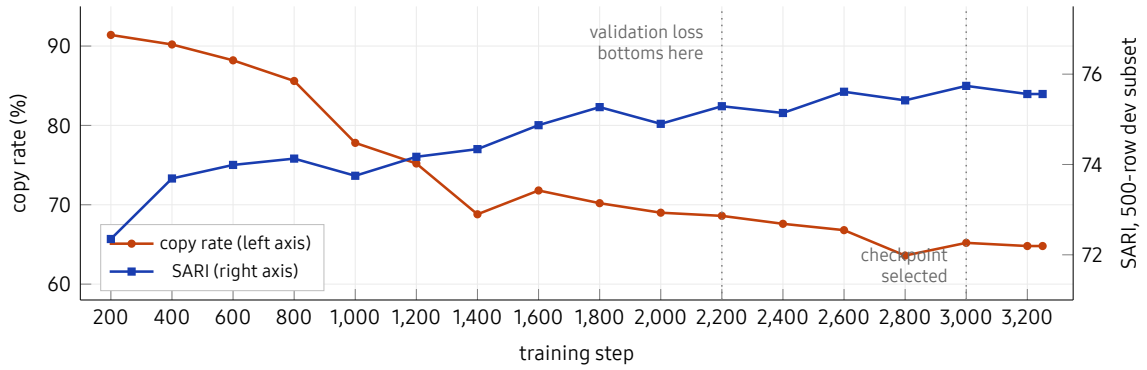


Figure 8. BayanSimplify-v0.1 training. The checkpoint was selected on a 0.18-point SARI difference while the model’s copying behaviour was still changing.

43.87 against a copy baseline of 45.05 (−1.19) and returns 77.7% of sentences unchanged: from SAMER’s novels it learned when not to edit, not how to simplify administrative prose.

Table 9. BayanSimplify-v0.1 on the locked BAREC test set (7,286 sentences, no reference simplifications), greedy decoding. Stripped: tashkeel removed from the source before the model sees it, as the application does.

Input	Copy rate	BERTScore	Readability drop	Source words kept	Rows losing >half
Stripped (as deployed)	68.6%	0.98	−0.04	94.9%	1.4%
Raw, with tashkeel	37.5%	0.91	−0.45	78.2%	17.5%

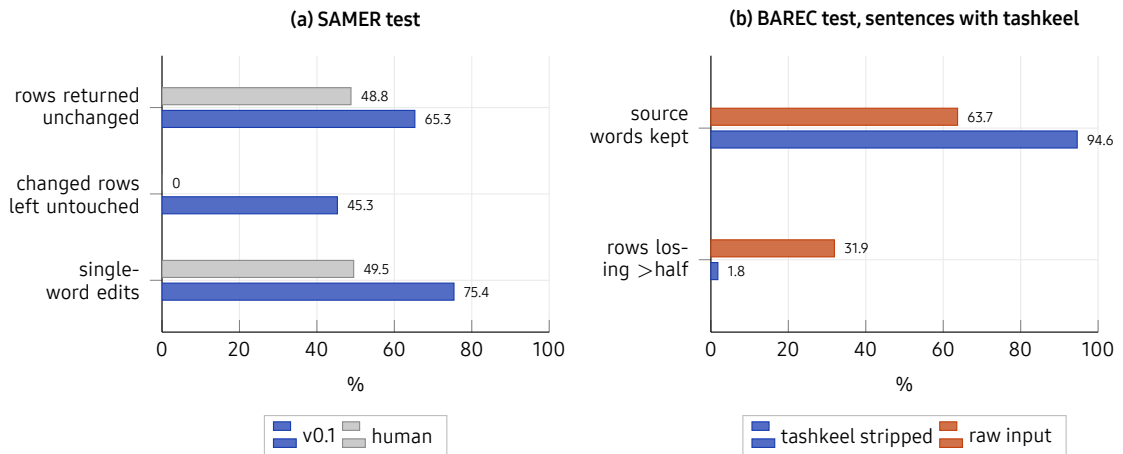


Figure 9. BayanSimplify-v0.1 behaviour. (a) SAMER test, greedy, against the editors’ references. (b) BAREC test, the 3,918 sentences that carry tashkeel, greedy, with and without tashkeel stripped.

Meaning errors. On dev, 832 of BayanSimplify-v0.1’s substitutions disagree with the editor, 347 of them on words the editor deliberately kept, and some reverse the meaning: عزلة (isolation) became استعباد (enslavement), ملجم (bridled) became مسلح (armed). SARI and BLEU are blind to this class of error, which is why BayanBench scores meaning separately from ease.

v0.2 training. Both runs clear BayanSimplify-v0.1’s selection score (Table 10, Figure 10); for reference, BayanSimplify-v0.1 scored 63.66 SARI on SAMER dev’s changed rows (all 1,754 of them, a superset of the 600 used here). AraT5v2 was still improving at its last evaluation; AraBART peaked at epoch 9 and had been flat since epoch 6. Copying falls from 80–91% of rows to about 51% for both, against 33.3% for the editors on the same rows: the recipe reduces

over-conservatism but does not remove it, and AraT5v2 still returns 186 of the 600 changed rows unchanged.

Table 10. The v0.2 runs, one T4 GPU each. SARI and copy rate on the 900-row selection sample (600 changed, 300 unchanged SAMER dev rows, tag [S0]). Speed: sentences per second on SAMER test sources, fp16, batch 64, greedy.

	AraT5v2	AraBART
Parameters	368M	139M
Training time	6,783 s	2,600 s
Best checkpoint (step of 14,210)	14,200	12,780
SARI, 600 changed rows	66.58	64.71
SARI, all 900 rows	75.64	73.62
Copy rate, 900 rows (editors: 33.3%)	51.0%	51.4%
Generation speed (sentences/s)	80.8 [†]	292.8
[†] measured on BayanSimplify-v0.1, the same architecture and size, in the same run.		

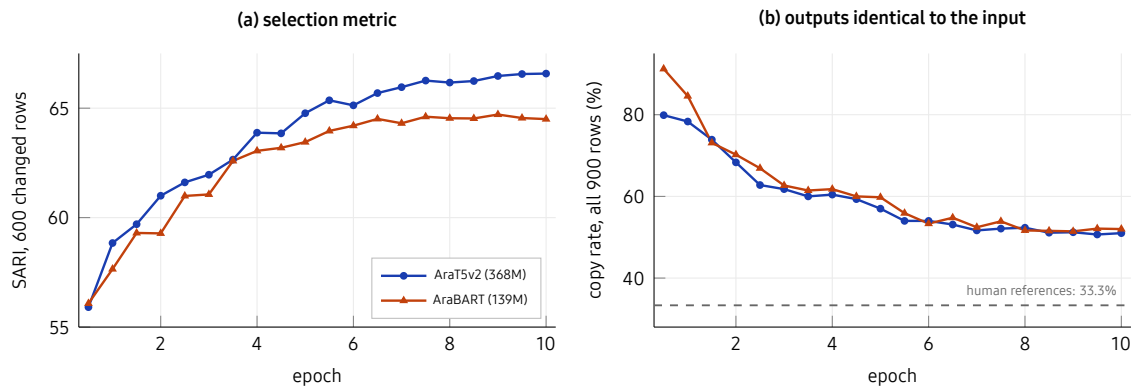


Figure 10. The v0.2 training, from the committed notebooks: (a) the selection metric at each of the 20 evaluations; (b) the share of outputs identical to the input, with the editors' rate as a dashed line.

3.4. Human Evaluation

Agreement between raters. On the 80 outputs given to every rater, the raters agreed moderately on meaning (Krippendorff's $\alpha = 0.51$, interval metric; 74% of rater pairs gave the same answer) and weakly on Arabic ($\alpha = 0.17$) and ease ($\alpha = 0.19$). The Arabic figure is low partly because almost every output was rated correct, so the raters agreed on 87% of pairs yet the few errors they flagged rarely coincided. Meaning is therefore the one human judgement firm enough to check an automatic scorer against; ease is reported per system but not used as a target.

The meaning scorer against people. Table 11 and Figure 11 compare the scorer's P(same meaning) with the raters' verdict on all 245 rated outputs. It separates the outputs the raters kept from the rest with an AUC of 0.85 [0.78–0.91], rising to 0.92 on the 80 outputs with three or more raters, where the human verdict is itself more reliable; it is 0.91 when a minor change counts as kept. Its scores are bimodal, so the cut-off matters less than the ranking. As a pass/fail rule at 0.5 it catches 42 of the 51 outputs the raters judged changed, including all 11 with a major change, and fails 53 of the 194 that the raters kept: the scorer is stricter than people, so “meaning kept” is a conservative rate. A cut-off of 0.3 would have failed seven fewer kept outputs and caught the same losses; we kept the 0.5 set before the ratings rather than tune the threshold on the sample used to check it.

Table 11. The open meaning scorer (Gemma 4 31B, P(same meaning)) against the human verdicts on the 245 rated single outputs. AUC: the chance that an output the raters kept scores higher than one they did not (0.5 = chance, 1 = perfect); 95% intervals from a bootstrap over documents.

Human verdict counted as “meaning kept”	Kept	AUC [95% CI]
Majority “same meaning”	194 of 245	0.85 [0.78–0.91]
Majority not “major change” (a minor change counts as kept)	234 of 245	0.91 [0.86–0.96]
Every rater “same meaning”	175 of 245	0.83 [0.76–0.89]
Majority “same meaning”, tasks with three or more raters	64 of 80	0.92 [0.86–0.97]
The scorer as a pass/fail rule (kept = majority “same meaning”)		
Threshold on P(same)	Losses caught	Kept outputs failed
0.3	42 of 51	46 of 194
0.5 (frozen)	42 of 51	53 of 194
0.7	45 of 51	61 of 194

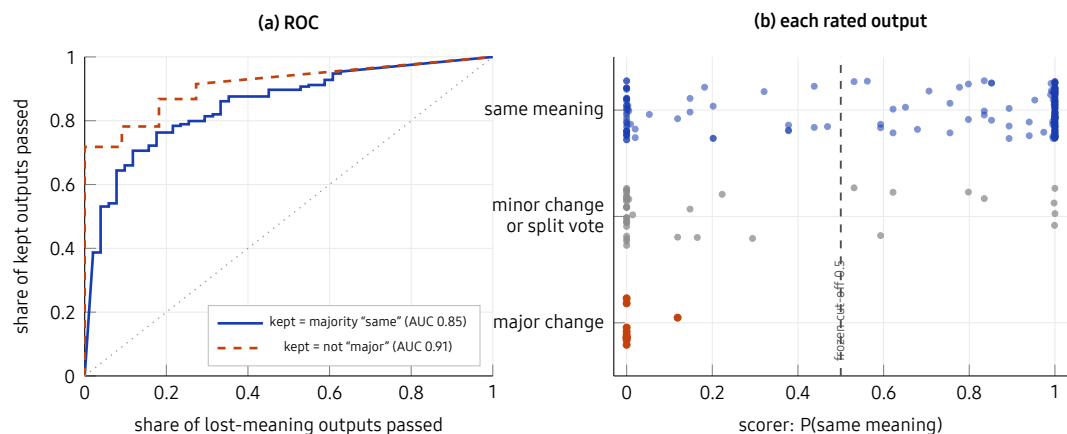


Figure 11. The meaning scorer against the raters, on 245 rated dev outputs. (a) ROC curves for two definitions of a kept meaning; the dotted diagonal is chance. (b) Every rated output by the raters’ majority answer (194 same, 40 minor or split, 11 major) against the scorer’s P(same); rows are jittered vertically for legibility.

Each system, as people rated it. Table 12 and Figure 12 show each system on the 35 changed outputs drawn evenly from it. The raters kept the meaning in 94% of the int8 AraT5v2 bundle’s rewrites and 89% of the untagged AraT5v2’s, against 71% for each of the three AraBART systems. The scorer, applied to the same outputs, also places every AraT5v2 system above every AraBART one, and is 6 to 22 points stricter than the raters on each system. The raters judged about half of every system’s rewrites easier (49–57%) and 89–97% of them correct Arabic. Because copies were not rated, these are rates among outputs that changed, not over all inputs as in subsection 3.5.

The reader with dyslexia. Table 13 gives the sixth rater’s answers. Of 30 rewrites, 17 were easier to read for them, 9 no different and 4 harder; their answer matched the team’s majority on 67% of the shared outputs, about as often as two team raters agreed with each other on ease (61%). The int8 AraT5v2 bundle, the most cautious system, was the only one never found easier. One reader is not a study, but the direction matches the benchmark: the system that keeps meaning best is also the one that simplifies least.

Side-by-side comparisons. Table 14 counts the team’s votes. The clearest contrast is architecture without a tag, where AraT5v2 was preferred in most votes; the other contrasts hold too few votes to separate the systems.

Table 12. Each system as the raters and the scorer saw the same outputs: the 35 changed outputs per system drawn evenly (copies of the input were not rated). Rates in %, a strict majority of the task's raters; 95% intervals from a bootstrap over documents. *Scorer*: $P(\text{same}) \geq 0.5$ on the same outputs.

System	Same meaning: raters	Same meaning: scorer	Easier: raters	Correct Arabic: raters
v0.2, int8 app	94 [85–100]	80 [67–92]	49 [30–67]	91 [82–100]
v0.2, no tag	89 [78–97]	83 [71–94]	54 [36–72]	89 [77–97]
v0.2, [s2]	77 [62–91]	63 [46–78]	49 [34–65]	97 [90–100]
v0.2-Fast, int8 app	71 [56–86]	51 [31–72]	54 [38–70]	94 [86–100]
v0.2-Fast, no tag	71 [52–88]	54 [38–71]	51 [33–68]	91 [82–100]
v0.2-Fast, [s2]	71 [57–86]	49 [34–63]	57 [43–72]	97 [92–100]

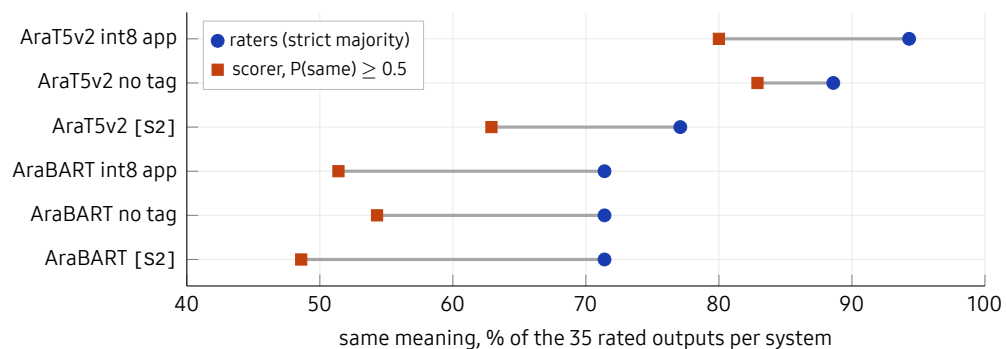


Figure 12. Same meaning, per system, on the same 35 outputs each: the raters' verdict (circle) and the scorer at $P(\text{same}) \geq 0.5$ (square).

The BayanSimplify-v0.3 round. A second round (4–5 October) rated the app's three models as the reader gets them: the fast model (AraBART int8), BayanSimplify-v0.2 int8 and BayanSimplify-v0.3 int8 with four beams, all through the app's text step. It used 18 Bayan-Bench test sentences (core and outside sets; no held-out item, so none of BayanSimplify-v0.3's training text) and was blind. A Latin square gave each of the 4 team raters every sentence once and each model on six, so each model has 24 team ratings on the same sentences; six comparisons, two per pair of models, were rated by everyone. BayanSimplify-v0.3 was rated best. It kept the meaning as often as any model (96% of ratings, 0% major changes; BayanSimplify-v0.2 96%, the fast model 92%) and was the most often found easier: 50% of its ratings [26–73] against 38% for BayanSimplify-v0.2 and 38% for the fast model, and on the outputs it changed, 86% against 60% and 45%, with none rated harder. In the side-by-side comparisons every team vote between BayanSimplify-v0.3 and BayanSimplify-v0.2 chose BayanSimplify-v0.3 (8 of 8), and BayanSimplify-v0.3 won 5 of the 8 votes against the fast model (1 for the fast model, 2 equal). The reader with dyslexia, who rated all 24 tasks, chose BayanSimplify-v0.3 in every comparison that included it (2 of 2 against BayanSimplify-v0.2, 2 of 2 against the fast model); on single outputs he found 4 of BayanSimplify-v0.3's 6 easier and none harder, against 2 of 6 for BayanSimplify-v0.2 and 5 of 6 for the fast model. The round is small, 18 sentences, so it confirms the direction of the benchmark rather than measuring a size.

Table 13. The reader with dyslexia: “was it easier for you?” on 30 outputs (five per system), against the original.

System	Easier	Same	Harder
v0.2, int8 app	0	3	2
v0.2, no tag	4	0	1
v0.2, [S2]	3	2	0
v0.2-Fast, int8 app	3	2	0
v0.2-Fast, no tag	5	0	0
v0.2-Fast, [S2]	2	2	1
All	17	9	4

Table 14. Side-by-side comparisons by the team raters: which version is better for a reader with dyslexia (easier, with the same meaning). Counts of rater votes.

Contrast	Systems (first vs second)	First	Tie	Second
app	v0.2-Fast, int8 app vs v0.2-Fast, [S2]	4	4	3
app	v0.2, int8 app vs v0.2, [S2]	6	2	8
architecture	v0.2-Fast, [S2] vs v0.2, [S2]	5	5	5
architecture	v0.2-Fast, no tag vs v0.2, no tag	2	1	10
tag	v0.2-Fast, [S2] vs v0.2-Fast, no tag	10	0	6
tag	v0.2, [S2] vs v0.2, no tag	3	7	6

3.5. BayanBench Results

RQ2: the v0.2 models on BayanBench. Each of the seven systems was run once on the Bayan-Bench splits: the v0.2 models on both architectures in the medium setting ([S2]) and without a tag, the two int8 bundles exactly as the app calls them, and copying the input. Table 15 reports the test split on core items; Table 16 isolates one change at a time, and Table 19 gives the other split. Five findings follow.

F1 Meaning and simplification trade off. The medium setting cuts 6.3 (AraT5v2) and 6.7 (AraBART) words from the longest clause and keeps the meaning in 59.5% and 40.1% of outputs. Without the tag the same models cut 4.5 and 4.9 words and keep 68.3% and 51.4%; the int8 AraT5v2 bundle cuts 1.9 and keeps 78.6% (Figure 13). Within each architecture, the tagged setting simplifies most and keeps meaning least often, and no system simplifies as much as the tagged setting while keeping meaning as often as the shipped bundle.

F2 AraT5v2 keeps meaning better at every setting. At the same setting AraT5v2 keeps the meaning 16.8 points more often than AraBART without a tag [+12.1, +21.6], 19.4 with the [S2] tag and 28.1 as an int8 bundle, every interval excluding zero; without a tag the two cut long clauses by similar amounts. The human ratings agree (Figure 12).

F3 Simplification is mostly structural. The predicted reading level falls by at most 0.40 levels on average, and each output removes 0.31–0.68 hard words on average, where a clause loses up to 6.7 words: the models shorten clauses far more than they change vocabulary.

F4 Restraint is weak. Easy text comes back unchanged in only 11.7–48.1% of C items, protected text keeps its letters in 4.4–47.8%, and a selection cut mid-sentence keeps its ending in 36.5–45.9% of E1 items; the rest are completed, sometimes with invented words (Table 18). No training corpus shows these behaviours: SAMER’s unchanged pairs are lit-

Table 15. The v0.2 models on BayanBench v2, **test** split, *core* items (749 of 1250 items; 197 documents in the split), each system run once. Meaning: the open scorer, Gemma 4 31B; a copy of the input keeps its meaning by definition. Simpler: mean change from selection to output, over the items where there is something to simplify. Rates in %, means with their sign; 95% intervals from a bootstrap over whole documents in brackets. The two [S2] columns send the medium strength tag the model was trained with; the two *int8 app* columns are the quantised bundles the app downloads, which send no tag. All outputs pass through the app's guards. Bold: the best model on each row. *n* = items the measure applies to.

Measure	Copy the input	v0.2, [S2]	v0.2-Fast, [S2]	v0.2, int8 app	v0.2-Fast, int8 app	<i>n</i>
Meaning						
Meaning kept (P(same) $\geq$ 0.5, numbers kept), %	100.0	59.5 [55–64]	40.1 [36–44]	78.6 [75–82]	50.5 [46–55]	659
Mean P(same meaning)	1.00	0.59 [0.55, 0.64]	0.40 [0.37, 0.45]	0.79 [0.75, 0.82]	0.51 [0.47, 0.56]	659
Simpler (mean change)						
Longest clause: words cut (clause over 15)	+0.0	+6.3 [+5.0, +7.9]	+6.7 [+5.4, +8.3]	+1.9 [+1.0, +2.9]	+4.8 [+3.5, +6.1]	212
Reading level: levels lower (level 10+)	+0.00	+0.40 [+0.31, +0.49]	+0.38 [+0.30, +0.47]	+0.14 [+0.08, +0.20]	+0.28 [+0.19, +0.38]	543
Hard words removed (items with any)	+0.00	+0.61 [+0.46, +0.81]	+0.68 [+0.54, +0.85]	+0.31 [+0.22, +0.44]	+0.59 [+0.46, +0.75]	328
Checks, %						
Every number kept	100.0	82.7 [65–95]	78.7 [63–90]	84.0 [70–94]	76.0 [59–88]	75
No house-rule break	100.0	99.8 [99–100]	99.7 [99–100]	100.0	100.0	659
Behaviour, %						
Easy text left unchanged (C)	100.0	14.3 [7–22]	11.7 [5–19]	48.1 [36–61]	24.7 [15–35]	77
Cut-off selection keeps its ending (E1)	100.0	36.5 [25–47]	38.8 [27–50]	38.8 [28–49]	45.9 [34–58]	85
Protected text: letters unchanged (F)	100.0	4.4 [0–10]	4.4 [0–11]	47.8 [30–70]	24.4 [16–36]	90
Flags (not failures), %						
Negation count changed	0.0	23.1 [17–30]	24.1 [17–32]	7.1 [4–11]	18.4 [13–25]	139

erary sentences, and no corpus contains protected or cut-off text. Numbers survive in 76.0–84.0% of the core items that contain one. The app's text step now handles protected text, numbers and cut-off endings in code (subsection 2.4), and BayanSimplify-v0.3 improves restraint (Table 3.5).

F5 The deployed format buys meaning with ease. The bundles the app ships send no strength tag (Table 16). Dropping the tag raises meaning kept by 8.8 (AraT5v2) and 11.4 (AraBART) points and cuts 1.9 and 1.8 fewer words from the longest clause. Quantising to int8 rewords most outputs (the app's AraT5v2 outputs match the float model's on only 234 of 731 dev items). For AraBART it makes no measurable difference to meaning kept. For AraT5v2 it moves the model toward copying (2.6 fewer words cut, 20.8 points more easy text left alone) and keeps the meaning 10.3 points more often [+7.1, +13.5] on this split, though not on dev, which makes the shipped AraT5v2 bundle the safest system and the least simplifying one.

Across item sets. Table 17 reports the four item sets side by side. Meaning is kept at similar

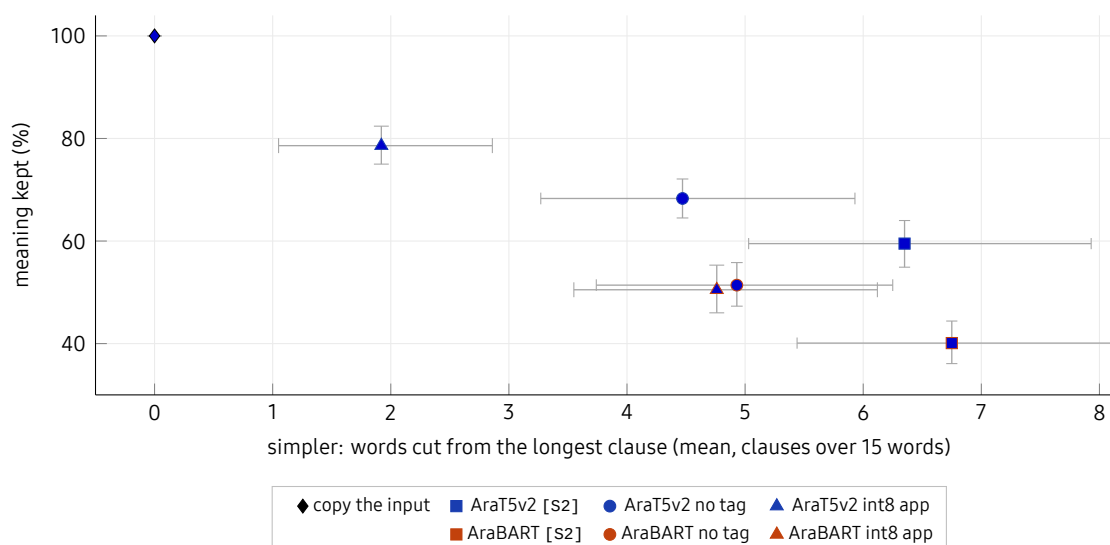


Figure 13. Meaning kept against simplification, test split, core items, with 95% document-bootstrap intervals. Copying the input sits at the top left by definition. Colour: the model; shape: the setting.

Table 16. Paired effects on the test split (core items): one change at a time, same items. Differences in points (rates) or words (clause), with 95% document-bootstrap intervals; * = the interval excludes zero.

One change	Meaning kept (points)	Words cut from the longest clause	Easy text unchanged (points)
AraT5v2: drop the [S2] tag	+8.8* [+5.9, +12.0]	-1.9* [-2.4, -1.4]	+13.0* [+4.8, +21.4]
AraBART: drop the [S2] tag	+11.4* [+7.0, +15.5]	-1.8* [-2.5, -1.2]	+11.7* [+3.4, +19.1]
AraT5v2: PyTorch → int8 app	+10.3* [+7.1, +13.5]	-2.6* [-3.3, -1.9]	+20.8* [+11.9, +31.1]
AraBART: PyTorch → int8 app	-0.9 [-3.0, +1.1]	-0.2 [-0.5, +0.2]	+1.3 [-3.2, +5.9]
[S2]: AraBART → AraT5v2	+19.4* [+14.9, +23.5]	-0.4 [-0.9, +0.2]	+2.6 [-6.8, +12.5]
No tag: AraBART → AraT5v2	+16.8* [+12.1, +21.6]	-0.5 [-1.1, +0.2]	+3.9 [-8.6, +16.7]
int8 app: AraBART → AraT5v2	+28.1* [+23.3, +32.6]	-2.8* [-3.7, -2.0]	+23.4* [+11.3, +36.9]

rates on text published after every corpus we use (outside) as on core, so the results do not rest on BAREC’s genres. Clause gains vary between sets without a common direction, and restraint is weak everywhere.

Error analysis. Table 18 shows test outputs of AraT5v2 in the medium setting. The first is the intended behaviour: a 17-word sentence split into two cause-linked clauses. The second passes every code check and reverses a quantity, which is why meaning needs a scorer and not code alone. The others are the failures of F4. One example is not printed: in a Qur’anic verse (*al-Kāfirūn* 109:4) the model dropped the negation ﻻ, inverting the verse’s meaning, the most serious error we observed and the reason protected text must never reach the model.

BayanSimplify-v0.3. BayanSimplify-v0.3 is AraT5v2 trained on corpus v1. Compared with BayanSimplify-v0.2 (medium setting), as float models on the behaviours BayanSimplify-v0.2 got wrong, it leaves easy text unchanged in 55% of cases against 14%, keeps protected text intact in 97.8% against 4.4%, and keeps numbers in 93% of cases. Because int8 had changed most of BayanSimplify-v0.2’s outputs, BayanSimplify-v0.3 was gated as it would ship: its int8 bundle, run through the app’s current text step (subsection 2.4), against the app’s large model at the time (BayanSimplify-v0.2, int8) through the same step, on the BayanBench core items. The rule, set before the run, was that meaning kept must not be significantly lower, the longest

Table 17. The four item sets on the test split, reported side by side and never averaged; number of items in brackets. *Core*: BAREC test documents; *held-out*: news and legal text; *outside*: web text published after every corpus we use; *written*: medicine-leaflet sentences written for the benchmark.

System and measure	Core	Held-out	Outside	Written
v0.2, [s2]				
Meaning kept, %	59.5 ⁽⁶⁵⁹⁾	64.6 ⁽²⁷⁴⁾	60.8 ⁽²⁰⁴⁾	65.2 ⁽²³⁾
Longest clause: words cut	+6.3 ⁽²¹²⁾	+5.8 ⁽⁵⁸⁾	+3.5 ⁽⁶⁵⁾	+2.0 ⁽¹⁾
Easy text left unchanged, %	14.3 ⁽⁷⁷⁾	6.5 ⁽⁹²⁾	6.1 ⁽³³⁾	—
v0.2-Fast, [s2]				
Meaning kept, %	40.1 ⁽⁶⁵⁹⁾	49.3 ⁽²⁷⁴⁾	37.7 ⁽²⁰⁴⁾	52.2 ⁽²³⁾
Longest clause: words cut	+6.7 ⁽²¹²⁾	+5.5 ⁽⁵⁸⁾	+4.9 ⁽⁶⁵⁾	+4.0 ⁽¹⁾
Easy text left unchanged, %	11.7 ⁽⁷⁷⁾	7.6 ⁽⁹²⁾	21.2 ⁽³³⁾	—
v0.2, int8 app				
Meaning kept, %	78.6 ⁽⁶⁵⁹⁾	77.0 ⁽²⁷⁴⁾	75.5 ⁽²⁰⁴⁾	82.6 ⁽²³⁾
Longest clause: words cut	+1.9 ⁽²¹²⁾	+2.3 ⁽⁵⁸⁾	+2.1 ⁽⁶⁵⁾	-1.0 ⁽¹⁾
Easy text left unchanged, %	48.1 ⁽⁷⁷⁾	22.8 ⁽⁹²⁾	27.3 ⁽³³⁾	—
v0.2-Fast, int8 app				
Meaning kept, %	50.5 ⁽⁶⁵⁹⁾	56.9 ⁽²⁷⁴⁾	46.6 ⁽²⁰⁴⁾	69.6 ⁽²³⁾
Longest clause: words cut	+4.8 ⁽²¹²⁾	+3.9 ⁽⁵⁸⁾	+4.2 ⁽⁶⁵⁾	+0.0 ⁽¹⁾
Easy text left unchanged, %	24.7 ⁽⁷⁷⁾	18.5 ⁽⁹²⁾	12.1 ⁽³³⁾	—

clause must shrink by more than the app's, and no check may be significantly worse. On dev, BayanSimplify-v0.3 passes: meaning kept differs by -2 points [-7, +4] (313 of 320 items scored), 4.3 more words are cut from the longest clause [+3.0, +5.7], the reading level falls 0.27 more [+0.15, +0.42], easy text is left unchanged 26 points more often, and every number is kept. On test, run once to confirm, the simplification holds (3.9 more words cut [+2.8, +5.0], reading level 0.13 lower [+0.04, +0.20], 0.26 more hard words removed [+0.17, +0.36]), but with greedy decoding meaning kept falls by 6 points [-10, -1], a significant loss (648 of 659 items scored for both).

Beam search makes BayanSimplify-v0.3 the best model we have measured. #61 (Khengari) showed on BayanSimplify-v0.3's int8 bundle that four beams keep the meaning 4–8 points more often than greedy decoding, and the app gained beam search with transformers' exact tokens (#62) as a setting, *More faithful, slower* (#66). Run with four beams by the app's own engine and through the app's text step, with every core item scored, BayanSimplify-v0.3 outperforms every model in Table 15: it keeps the meaning in 82% of test outputs, against at most 78.6%, and cuts 5.2 words from the longest clause, more than any of them except the tagged settings, which keep the meaning in at most 59.5%. Even against BayanSimplify-v0.2 run behind the same newest checks it is no worse on meaning (-3 points on test [-7, +1], +1 on dev [-4, +7]) and simplifies far more: 3.4 more words cut from the longest clause on test [+2.4, +4.4] (3.8 on dev [+2.6, +5.1]), easy text left unchanged 18 points more often (28 on dev), and every number kept. Against its own greedy decoding, four beams keep the meaning 3 points more often on test [+0, +5] (+3 on dev [-1, +7]) at no significant cost in clause length [-1.3, +0.3].

Khengari's benchmark report (#57, #58), which scores 35 systems with every number generated from the bench's own scorecards, reaches the same verdict and recommends BayanSimplify-v0.3 app int8 with four beams and the safety net as the system to ship. On the test core items, against BayanSimplify-v0.2 in the app as benchmarked in Table 15, it keeps every number (100% against 84%), changes no negation, limit or condition word, leaves easy text unchanged

Table 18. Test outputs of AraT5v2 ([s2]), from the BayanBench test split. Glosses are ours.

Track	Source	Output	What happened
B	فالمهم الصفح والتسامح وذلك للتخفيف على الصحة النفسية للفرد ولانتشار المحبة والتعاون بين أفراد المجتمع بشكل عام.	فالمهم الصفح والتسامح؛ لأن ذلك يساعد على تحسين الصحة النفسية للفرد، ويساعد على انتشار المحبة والتعاون بين أفراد المجتمع بشكل عام.	Split into two clauses joined by a stated cause; meaning kept.
B	منذ قرنين على الأكثر بدأ الإنسان يدخل دائرة حلزونية متصاعدة متفاقة بين الطلب والاستجابة للطلب ...	منذ قرنين على الأقل، بدأ الإنسان يدخل دائرة حلزونية متصاعدة بين الطلب والاستجابة له ...	“At most two cen- turies” became “at least”: a reversal every code check passes.
C	يعتبر منصب الرئيس شاغرا في أي من الحالات التالية:	يعتبر منصب الرئيس شاغرا في أي حالة:	Easy legal text edited; “in any of the follow- ing cases” lost its list.
E1	الشكل المنطقي الأساسي في علم المنطق، ويتم تحديد صحة المفاهيم من خلال	الشكل المنطقي الأساسي في علم المنطق، ويتم تحديد صحة المفاهيم من خلال اختبار صحة المفاهيم	A cut-off selection completed with in- vented words and a full stop.
A2	... بنتيجة (١/٣) في المباراة ...	... بنتيجة (/) في المباراة ...	Persian-range digits dropped.

in 74% of cases against 48%, and cuts 3.5 more words from the longest clause [+2.5, +4.5], while meaning kept rises by 4 points [+0, +8].¹ The same report shows where the remaining headroom lies. The phone version simplifies less than the float model: float BayanSimplify-v0.3 with the net cuts 7.7 words from the longest clause and lowers the reading level by 0.38, the int8 bundle with four beams and the net 5.4 words and 0.20 levels. BayanSimplify-v0.3 also leaves 47% of items unchanged against the app’s 19%, and on the outputs it does change, P(same) is 0.67 against the app’s 0.74: it rewrites less often, but harder. BayanSimplify-v0.3 with four beams is therefore the app’s large model (5 October). Its four beams take about three times as long per sentence as greedy decoding (Table 21); a reader who prefers word-by-word streaming can turn the setting off, and BayanSimplify-v0.2 stays available as the earlier large model.

3.6. User Interface

Figure 14 shows the interface on the phone. Selecting Arabic text in any app offers تبسيط (*Simplify*) among the selection actions; choosing it opens the result over the app, the rewrite appearing sentence by sentence as it is decoded. One toggle switches between *Simplified* and *Original*; below it are read-aloud, copy and share, and the current word is highlighted while Bayan reads. The panel can be minimised to a small window over the page that keeps following the reading and moves by its handle, with play and stop also in the phone’s media controls, so the reader keeps the host app in view while listening. The app follows Android’s Material 3 design and takes its colours from the phone’s Material You palette rather than a design system of its own, and sets Arabic right to left without justification, because justification in Arabic stretches words and opens uneven gaps. Settings hold the reader’s typeface, size (22 sp by default), line height (1.8) and word spacing, the one spacing adjustment that carries over to cursive Arabic, together with the background, the model, the voice and its speed, and word

¹His report puts the system at 83% meaning kept and 5.4 words cut on test; this section’s run gives 82% and 5.2. Both decode the same int8 bundle with four beams; his applies the safety net to transformers’ outputs on another CPU, where int8 rounding differs slightly, while ours runs the app’s own decoder and its full text step (sentence pieces, the guards, punctuation repair). The comparisons they support point the same way.

Table 19. The v0.2 models on BayanBench v2, **dev** split, *core* items (380 of 731 items; 129 documents in the split), each system run once. Meaning: the open scorer, Gemma 4 31B; a copy of the input keeps its meaning by definition. Simpler: mean change from selection to output, over the items where there is something to simplify. Rates in %, means with their sign; 95% intervals from a bootstrap over whole documents in brackets. The two [S2] columns send the medium strength tag the model was trained with; the two *int8 app* columns are the quantised bundles the app downloads, which send no tag. All outputs pass through the app's guards. Bold: the best model on each row. *n* = items the measure applies to.

Measure	Copy the input	v0.2, [S2]	v0.2-Fast, [S2]	v0.2, int8 app	v0.2-Fast, int8 app	<i>n</i>
Meaning						
Meaning kept (P(same) $\geq$ 0.5, numbers kept), %	100.0	62.5 [57–68]	41.9 [36–49]	78.4 [73–83]	54.7 [49–61]	320
Mean P(same meaning)	1.00	0.63 [0.58, 0.68]	0.42 [0.36, 0.49]	0.79 [0.74, 0.84]	0.55 [0.49, 0.61]	320
Simpler (mean change)						
Longest clause: words cut (clause over 15)	+0.0	+6.4 [+5.2, +7.6]	+6.0 [+5.0, +6.9]	+1.4 [+0.6, +2.3]	+3.8 [+2.8, +4.7]	106
Reading level: levels lower (level 10+)	+0.00	+0.20 [+0.07, +0.35]	+0.25 [+0.11, +0.38]	+0.01 [−0.04, +0.07]	+0.21 [+0.10, +0.31]	255
Hard words removed (items with any)	+0.00	+0.58 [+0.33, +0.83]	+0.46 [+0.28, +0.66]	+0.34 [+0.19, +0.49]	+0.47 [+0.29, +0.65]	148
Checks, %						
Every number kept	100.0	76.0 [59–91]	72.0 [56–86]	72.0 [50–88]	72.0 [52–88]	25
No house-rule break	100.0	99.7 [99–100]	99.7 [99–100]	100.0	99.7 [99–100]	320
Behaviour, %						
Easy text left unchanged (C)	100.0	19.1 [8–29]	12.8 [5–21]	40.4 [26–57]	21.3 [11–31]	47
Cut-off selection keeps its ending (E1)	100.0	36.7 [24–49]	44.9 [32–57]	49.0 [38–59]	55.1 [42–67]	49
Protected text: letters unchanged (F)	100.0	11.7 [4–26]	6.7 [0–16]	48.3 [21–91]	26.7 [11–55]	60
Flags (not failures), %						
Negation count changed	0.0	20.4 [9–31]	18.0 [8–28]	2.1 [0–6]	2.1 [0–6]	47

highlighting, with a sample sentence that shows each change as it is made.

Table 20. What the measurements changed.

Finding	Change
Copying scores SARI 77.5 on SAMER test	Copy baseline and changed/unchanged split reported with every score
BayanSimplify-v0.1 collapses on diacritized input	Tashkeel stripped before the simplifier, in training and in the app
BayanSimplify-v0.1 chosen on SARI noise over a copy-heavy dev set	The v0.2 models select on changed-row SARI only
BayanSimplify-v0.1 over-edits unchanged rows yet copies 65% of all rows	All unchanged pairs kept in v0.2 training
SAMER teaches word substitution; BayanSimplify-v0.1 below copying on DAASI	DAASI and meaning-filtered external pairs added, each under its own tag
LLM-written external targets often summarise	Meaning filter calibrated on human rewrites
Prediction script omitted the training prefix	Prefix a required setting of every model and bundle
Length grouping shuffled the evaluation set	Grouping off; alignment check stops the run
SARI is blind to meaning errors	BayanBench: meaning scored separately from ease, with code checks
The closed judge was quota-bound, pass/fail and not rerunnable	Open meaning scorer, continuous, rerun by anyone, checked against six raters
Some BayanBench v1 measures did not track human labels	Dropped; v2 keeps only validated measures
Under transformers 5 the models' outputs changed silently	Prediction refuses to run under version 5
The v0.2 models over-edit easy text and protected text	Corpus v1 and BayanSimplify-v0.3; the app never sends scripture or set poetry to the model
The model drops numbers and flips negations	The app keeps the source sentence when a rewrite loses a number, a Latin word, a negation or a limit
The model invents text around headings and fragments	Sentences under six words pass through unchanged in the app

3.8. On the Device

RQ3: cost on the phone. Both simplifiers were exported to ONNX with the tokenizer inside the bundle and quantised to int8 (Table 21). The AraBART bundle, the default, fits the 250 MB budget. The AraT5v2 bundles are optional downloads of 471 MB: 46% of their weights are two untied 110k-row embedding matrices, and we kept them whole rather than trade quality for size. We tested both bundles in the app on a Xiaomi Mi 11X (Snapdragon 870, Android 13, 6 GB RAM), a phone that now sits in the mid-range, with the same two-sentence paragraph and six cold starts each. Because output streams, the reader waits for the first token, not the whole sentence: with AraBART 82–172 ms for a paragraph's first sentence and 28–30 ms for each later one, with AraT5v2 177–318 ms and 55–85 ms. AraBART loads in 1.0–1.7 s against 2.6–7.3 s, and finishes a sentence about twice as fast, the same ratio as on the laptop. BayanSimplify-v0.3 has the same architecture and bundle size as the AraT5v2 bundle measured here. With the beam setting on it decodes with four beams, which runs the decoder on four rows at a time and gives up streaming: over 120 BayanBench dev sentences it takes 1.19 s per sentence (median; 90th percentile 2.43 s) on the phone, against 0.41 s when the same bundle decodes greedily. Quantisation is not neutral for AraT5v2: int8 changed 62–74% of its outputs against the float model, though not the rate at which meaning is kept (F5). We did not prune their vocabulary, since any change to the shipped weights would need the whole evaluation again; the bundles are 449 MiB and 212 MiB on disk, the 471 MB and 222 MB of Table 21 in decimal units.

Table 21. On-device cost. Laptop: 4 CPU threads, median and 90th percentile per BayanBench test sentence, the app’s own decoding loop. Phone: Xiaomi Mi 11X (Snapdragon 870, Android 13, 6 GB RAM), from the app’s logs over six cold starts each on the same two-sentence paragraph (AraT5v2 on 26 September, AraBART on 3 October).

	AraBART, int8	AraT5v2, int8
Download (encoder + decoder + tokenizer)	222 MB (212 MiB)	471 MB (449 MiB)
Laptop CPU, per sentence (median / p90)	0.10 / 0.24 s	0.20 / 0.43 s
Phone: model load, cold	1.0–1.7 s	2.6–7.3 s
Phone: first token, first sentence	82–172 ms	177–318 ms
Phone: first token, later sentences	28–30 ms	55–85 ms
Phone: whole sentence	0.23–1.10 s	0.48–2.24 s
Phone: BayanSimplify-v0.3 per sentence, greedy / 4 beams (median, 120 dev sentences)	—	0.41 / 1.19 s
Read-aloud voice (Nabra-7M-Distill, int8)	8.7 MB	

Diacritization. Four systems were benchmarked on the 743-sentence Tashkeela test fixture (Table 22). Libtashkeel was selected: CATT ties it on accuracy (0.01 points, about six characters in 67,576) but is 16 times larger and carries a non-commercial licence, and CAMEL Tools changed the base letters of 32 of 100 sentences, corrupting the text it was asked to vocalise. Before shipping it we re-ran the benchmark with the same fixture and scoring code (docs/diacritization_comparison.md). Libtashkeel, CATT and Mishkal reproduce the table exactly. Newer models did not displace it: the rawi models rewrite hamza seats in 97–106 of the 743 sentences, which a diacritizer must never do; Tashkeel-v3 scores 8.21% on CATT’s own scorer, not the 4.98% its card reports, at 171M parameters; and Tashkeel-v4 is released for research use only. On CATT’s scorer CATT and Tashkeel-v3 lead Libtashkeel by up to 1.4 points, at 16 and 140 times its size.

Table 22. Diacritization benchmark. DER = diacritic error rate; base-letter corruption measured on 100 sentences.

System	DER	DER, no case	WER	Base letters changed	Size	Licence
CAMEL Tools MLE [39]	16.57%	13.97%	56.73%	32%	129 MB	MIT/GPL
CATT [40]	8.44%	6.92%	29.40%	0%	77.9 MB	CC BY-NC
Mishkal [41]	12.74%	10.12%	39.33%	0%	4.3 MB	GPLv3
Libtashkeel [42]	8.43%	6.91%	29.57%	0%	4.8 MB	MIT

4. Projected Impact

4.1. Accomplishments and Benefits

The answers to the research questions are these. **RQ1:** no. On copy-heavy test sets, SARI rewards inaction and is blind to reversals of meaning; a model that beats copying on the sentences editors changed still scores below copying overall, and what it learned from SAMER is vocabulary substitution without restructuring. An open meaning scorer, by contrast, agrees with human raters (AUC 0.85) and can be rerun by anyone. **RQ2:** partly. The models lower linguistic load, but not for free: every setting trades meaning for simplification, the safest one simplifies least, and AraT5v2 keeps the meaning of 59.5–78.6% of outputs, more than AraBART. Removing the strength tag helps meaning, so the untagged bundles the app ships are safer

Table 23. Objectives from subsection 1.2, against the evidence.

Objective	Evidence	Outcome
O1 Beat copying on changed rows; lower linguistic load without losing meaning	BayanSimplify-v0.1: 61.83 vs 56.06 on SAMER test (+5.77). The v0.2 models lower clause length and hard words, but keep the meaning in 40.1–78.6% of outputs, and raters kept the meaning of 71–94% of changed outputs. BayanSimplify-v0.3 with four beams outperforms them: 82% meaning kept on test, 5.2 words cut from the longest clause	partly met
O2 Diacritize without changing letters	Libtashkeel in the app (a setting): DER 6.91% without case endings, no letter changed in 743 + 300 sentences, 4.8 MB	met
O3 Run on a phone; default within 250 MB	Both bundles run on a Snapdragon 870 phone, first token in 28–318 ms; the default AraBART int8 is 222 MB; the optional AraT5v2 int8 bundles (BayanSimplify-v0.3, the large model, and BayanSimplify-v0.2) are 471 MB, kept whole for quality	met
O4 Evidence from readers	A reader with dyslexia rated 30 rewrites for ease, with informed consent: 17 easier, 4 harder. No one with dyslexia has used the app, and there is no reading-time or comprehension study	started

than the tagged settings. BayanSimplify-v0.3 with four beams moves the trade-off: it keeps the meaning more often than any earlier model and simplifies more than BayanSimplify-v0.2 in the app. **RQ3:** yes for AraBART, in part for AraT5v2. Streamed output reaches the reader in under a third of a second on a mid-range phone with either bundle (BayanSimplify-v0.3 with the beam setting on shows a whole sentence in about 1.2 s); the default int8 AraBART bundle fits the 250 MB budget, the optional AraT5v2 bundles, BayanSimplify-v0.3's included, take 471 MB, and int8 changes most of AraT5v2's outputs against the float model.

Benefits. For readers: a tool that offers a simpler version where and when the difficulty occurs, offline and private, with the original always one tap away; we have not yet tested whether it helps readers with dyslexia. For schools: a tool that runs on the phones students already own. For the field: a documented case of test-set contamination in a public Arabic simplification corpus, an evaluation protocol that exposes copy-heavy scores, and a benchmark and a meaning judge, checked against people, that score behaviours rather than agreement with one editor.

4.2. Future Improvements

The evaluation points to specific next steps. The central one is to move the trade-off of F1: a model that simplifies as much as the tagged setting while keeping meaning as often as the shipped bundle. The meaning scorer, now checked against people, can rank candidate rewrites and select checkpoints, so that training is steered by the behaviours readers need rather than by SARI. BayanSimplify-v0.3 with four beams is the first step: it keeps BayanSimplify-v0.2's meaning rate in the app while cutting 5.2 words from the longest clause, short of the tagged setting's 6.3. Making that quality the default needs faster beam decoding on the phone, since four beams take about three times as long per sentence as greedy. The AraT5v2 bundles could be made smaller by pruning their vocabulary to the pieces Arabic text uses (46% of their parameters sit in two embedding matrices; about 223 MB pruned), but only if the full evaluation shows no loss. Sentence splitting needs training data that teaches it, which no human-written corpus provides. Last, a reading study with more readers with dyslexia, measuring reading time and comprehension, would turn ratings into evidence about reading.

4.3. Limitations

- **One reader with dyslexia.** The reader rated rewrites on a web page; nobody with dyslexia has yet used the app or been measured reading with it, and the five team raters do not have dyslexia.
- **The meaning scorer is a model.** It agrees with people at AUC 0.85 but is stricter than them, failing 53 of the 194 outputs they kept, so “meaning kept” is a conservative rate. It was checked on dev outputs of the six v0.2 systems, from 64 documents; a very different system could be judged differently.
- **Ease and Arabic are weakly agreed.** Raters agreed on ease and on correct Arabic with α of only 0.19 and 0.17, so the benchmark measures ease through clause length, reading level and hard words, and checks Arabic only through house rules.
- **The clause measure credits commas.** A rewrite can shorten its longest clause by adding commas as well as by splitting sentences, which is why it is always read next to meaning.
- **No reference-based test score for the v0.2 models.** They are compared with published work only through its dev selection score; its test evaluation is BayanBench, which has no human references.
- **Some tracks are small.** E3 and E4 hold 64 and 40 test items, and few core items contain a number (Table 15), so these rates separate only large differences.
- **One phone.** On-device figures come from one mid-range phone; the optional AraT5v2 bundles (471 MB) need more storage than the default.
- **BayanSimplify-v0.3’s meaning rests on the judge.** With four beams every test core item is scored, but no significant difference is not proof of none: the interval on test runs from -7 , $+1$ points. The human rating of BayanSimplify-v0.3 covers 18 sentences.
- **One run per configuration.** Each model was trained once, with one seed, so the comparison between architectures carries no estimate of training variance.

4.4. Conclusion

Bayan shows that Arabic text can be simplified entirely on a mid-range phone, from inside any app, at interactive latency, and that the reduction in linguistic load can be measured honestly. Getting there depended less on model size than on evaluation. The field’s standard score hid both what a model had learned and what it broke. And the behaviours a reader needs — shorter clauses, familiar words, the same meaning, and restraint where the text is already easy — had to be measured one by one before they could be improved. BayanBench makes those measurements reproducible by anyone on the team, its meaning scorer agrees with human raters, and its results set the next goal plainly: simplify as much as the boldest setting while keeping meaning as well as the safest.

References

- [1] A. F. Aldakhil. Prevalence of developmental dyslexia among primary school children in Arab countries: a systematic review and meta-analysis. *Research in Developmental Disabilities*, 152:104812, 2024. doi:10.1016/j.ridd.2024.104812
- [2] E. Saiegh-Haddad and H. Taha. The role of morphological and phonological awareness in the early development of word spelling and reading in typically developing and disabled Arabic readers. *Dyslexia*, 23(4):345–371, 2017. doi:10.1002/dys.1572

- [3] L. Rello, R. Baeza-Yates, L. Dempere-Marco and H. Saggion. Frequent words improve readability and short words improve understandability for people with dyslexia. *INTERACT*, pp. 203–219, 2013. doi:10.1007/978-3-642-40498-6_15
- [4] M. Rivero-Contreras, P. E. Engelhardt and D. Saldaña. An experimental eye-tracking study of text adaptation for readers with dyslexia: effects of visual support and word frequency. *Annals of Dyslexia*, 71(1):170–187, 2021. doi:10.1007/s11881-021-00217-1
- [5] I. A. Asadi, A. Asli-Badarneh and V. Vaknin-Nusbaum. The impact of morphological density on reading comprehension in Arabic: a comparison between typically developing and children with reading disabilities. *Dyslexia*, 30(1):e1761, 2024. doi:10.1002/dys.1761
- [6] L. Javourey-Drevet, S. Dufau, T. François, N. Gala, J. Ginestíe and J. C. Ziegler. Simplification of literary and scientific texts to improve reading fluency and comprehension in beginning readers of French. *Applied Psycholinguistics*, 43(2):485–512, 2022. doi:10.1017/S014271642100062X
- [7] E. W. Hermena, D. Drieghe, S. Hellmuth and S. P. Liversedge. Processing of Arabic diacritical marks: phonological–syntactic disambiguation of homographic verbs and visual crowding effects. *JEP: Human Perception and Performance*, 41(2):494–507, 2015. doi:10.1037/xhp0000032
- [8] A. Madjidi and C. Crick. Enhancing textual accessibility for readers with dyslexia through transfer learning. *ASSETS*, pp. 1–5, 2023. doi:10.1145/3597638.3614473
- [9] M. Gharaibeh and Z. Basulayyim. ChatGPT enhances reading comprehension for children with dyslexia in Arabic language. *Disability and Rehabilitation: Assistive Technology*, 20(8):2858–2871, 2025. doi:10.1080/17483107.2025.2508389
- [10] B. Alhafni, R. Hazim, J. D. Piñeros Liberato, M. Al Khalil and N. Habash. The SAMER Arabic Text Simplification Corpus. *LREC-COLING*, pp. 16079–16093, 2024.
- [11] K. Elmadani, N. Habash and H. Taha. A Large and Balanced Corpus for Fine-grained Arabic Readability Assessment. *Findings of ACL*, pp. 16376–16400, 2025.
- [12] Baseet: multi-level Arabic text simplification with Aya-8B-generated targets and AraT5v2/AraBART baselines. Graduation project, University of Jeddah, 2026. <https://github.com/raya-y/Baseet>
- [13] DAASI: parallel Arabic simplification of government and insurance text, GPT-4o drafts verified by two annotators. tabsIT project, CCO. <https://github.com/Huseen1141/tabsit>
- [14] T. Zerrouki and A. Balla. Tashkeela: Novel corpus of Arabic vocalized texts, data for auto-diacritization systems. *Data in Brief*, 11:147–151, 2017.
- [15] C. Scarton and L. Specia. Learning simplifications for specific target audiences. *ACL*, pp. 712–718, 2018. doi:10.18653/v1/P18-2113
- [16] L. Martin, É. de la Clergerie, B. Sagot and A. Bordes. Controllable sentence simplification. *LREC*, 2020.
- [17] L. Martin, A. Fan, É. de la Clergerie, A. Bordes and B. Sagot. MUSS: Multilingual unsupervised sentence simplification by mining paraphrases. *LREC*, pp. 1651–1664, 2022.
- [18] W. Xu, C. Napoles, E. Pavlick, Q. Chen and C. Callison-Burch. Optimizing statistical machine translation for text simplification. *TACL*, 4:401–415, 2016.
- [19] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger and Y. Artzi. BERTScore: Evaluating Text Generation with BERT. *ICLR*, 2020.
- [20] F. Alva-Manchego, C. Scarton and L. Specia. The (un)suitability of automatic evaluation metrics for text simplification. *Computational Linguistics*, 47(4):861–889, 2021. doi:10.1162/coli_a_00418
- [21] A. Devaraj, W. Sheffield, B. C. Wallace and J. J. Li. Evaluating factuality in text simplification. *ACL*, pp. 7331–7345, 2022. doi:10.18653/v1/2022.acl-long.506
- [22] L. Cripwell, J. Legrand and C. Gardent. Simplicity level estimate (SLE): a learned reference-less metric for sentence simplification. *EMNLP*, pp. 12053–12059, 2023. doi:10.18653/v1/2023.emnlp-main.739
- [23] D. Heineman, Y. Dou, M. Maddela and W. Xu. Dancing between success and failure: edit-level simplification evaluation using SALSA. *EMNLP*, pp. 3466–3495, 2023. doi:10.18653/v1/2023.emnlp-main.211
- [24] M. T. Ribeiro, T. Wu, C. Guestrin and S. Singh. Beyond accuracy: behavioral testing of NLP models with CheckList. *ACL*, pp. 4902–4912, 2020.
- [25] A. M. Bean et al. Measuring what matters: construct validity in large language model benchmarks. *NeurIPS Datasets and Benchmarks*, 2025.

- [26] A. Reuel, A. Hardy, C. Smith, M. Lamparth, M. Hardy and M. J. Kochenderfer. BetterBench: assessing AI benchmarks, uncovering issues, and establishing best practices. *NeurIPS Datasets and Benchmarks*, 2024.
- [27] E. Miller. Adding error bars to evals: a statistical approach to language model evaluations. arXiv:2411.00640, 2024.
- [28] D. Li et al. Preference leakage: a contamination problem in LLM-as-a-judge. *ICLR*, 2026.
- [29] K. Krippendorff. *Content Analysis: An Introduction to Its Methodology*, 4th ed. SAGE, 2019.
- [30] Google DeepMind. Gemma 4, 31B instruction-tuned, quantisation-aware int4 checkpoint (google/gemma-4-31B-it-qat-w4a16-ct). Hugging Face, 2026.
- [31] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. *SOSP*, pp. 611–626, 2023.
- [32] E. M. B. Nagoudi, A. Elmadany and M. Abdul-Mageed. AraT5: Text-to-Text Transformers for Arabic Language Generation. *ACL*, pp. 628–647, 2022.
- [33] A. Elmadany, E. M. B. Nagoudi and M. Abdul-Mageed. Octopus: A Multitask Model and Toolkit for Arabic Natural Language Generation. *ArabicNLP*, pp. 232–243, 2023.
- [34] M. K. Eddine, N. Tomeh, N. Habash, J. Le Roux and M. Vazirgiannis. AraBART: a Pretrained Arabic Sequence-to-Sequence Model for Abstractive Summarization. *WANLP*, pp. 31–42, 2022.
- [35] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li and P. J. Liu. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *JMLR*, 21(140):1–67, 2020.
- [36] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov and L. Zettlemoyer. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. *ACL*, pp. 7871–7880, 2020.
- [37] D. Samuel, A. Kutuzov, S. Touileb, E. Velldal, L. Øvrelid, E. Rønningstad, E. Sigdel and A. Palatkina. Nor-Bench – A Benchmark for Norwegian Language Models. *NoDaLiDa*, pp. 618–633, 2023.
- [38] L. Burchell et al. An Expanded Massive Multilingual Dataset for High-Performance Language Technologies (HPLT). *ACL*, pp. 17452–17485, 2025.
- [39] O. Obeid et al. CAMEL Tools: An Open Source Python Toolkit for Arabic Natural Language Processing. *LREC*, pp. 7022–7032, 2020.
- [40] abjadai. CATT: Character-based Arabic Tashkeel Transformer. <https://github.com/abjadai/catt>
- [41] T. Zerrouki. Mishkal: Arabic text vocalization. <https://github.com/linuxscout/mishkal>
- [42] Libtashkeel, an ONNX Arabic diacritization model (MIT), via the text2tashkeel wrapper.

5. Team Member Review and Comment



**Marwan
Elamami**



**Abdulrahman
Khengari**



**Ahmed
Alaeb**



**Sanad
Ali**



**Mohammed
Thabet**



**Abdul Majid
Mraied**

Team Cogni

NAME	REVIEW and COMMENT
Marwan Elamami	Leading Bayan taught me that the hard part of an AI product is knowing whether it works. I am proudest that we built a benchmark for the task, checked it against people, and let the numbers change our decisions, which is how BayanSimplify-v0.3 with four beams became the app's large model. Next, I want readers with dyslexia to use the app and to measure their reading itself.
Abdulrahman Khengari	I built the data side, from the first synthetic corpus to corpus v1, the meaning judge and BayanSimplify-v0.3, and the benchmark report that compared 35 systems on the same rules. Measuring every variant the same way is what showed that BayanSimplify-v0.3 with four beams is our best model.
Ahmed Alaeb	Training BayanSimplify-v0.1 and the v0.2 models showed me how much the data decides: SAMER taught word swaps, while the corpora we built taught the model to restructure. I am glad the model work ended in an app that runs offline on an ordinary phone.
Sanad Ali	I benchmarked the diacritizers and chose Libtashkeel, which now adds tashkeel in the app without changing a letter, and trained BayanSimplify-v0.2-Fast (AraBART), whose speed made it the fast default.
Mohammed Thabet	Building the Android side, from the selection menu and the result sheet to model downloads and typography settings, showed me how much a reading tool depends on small details: spacing, clear states, and never losing the reader's place.
Abdul Majid Mraied	My part was the evaluation groundwork: scoring and leakage scripts, the locked test sets and the copy baselines. They make sure a score in this report measures what it claims to.

6. Instructor Review and Comment

CATEGORY	SCORE	REVIEW and COMMENT
IDEA	__/10	
APPLICATION	__/30	
RESULT	__/30	
PROJECT MANAGEMENT	__/10	
PRESENTATION & REPORT	__/20	
TOTAL	__/100	